

User Manual

Battery testing with ZKE EBC testers – stand-alone, recorded on SD, operated from the browser

Firmware	1.4.72 (2 November 2026)
Hardware	EBCTester board, revision 1 (schematic as of 2 November 2026)
Supported testers	ZKE EBC-A20, EBC-A40L, EBC-A50L
Document	Edition November 2026, English
Project	github.com/Gemelon/EBCTester

© 2026 Thomas Stoll. Software and documentation licensed under the Apache License 2.0. EBCTester is an independent hobby project and not a product of, or supported by, ZKETECH.

Contents

Part I – Operating Manual . . . 4

1 **Introduction** . . . 5
- 1.1 What EBCTester does . . . 5
- 1.2 Main features . . . 5
- 1.3 Supported testers . . . 6
- 1.4 What you need . . . 6
- 1.5 Safety information . . . 6

2 **Setting up** . . . 8
- 2.1 Connecting the hardware . . . 8
- 2.2 First start and login . . . 8
- 2.3 Connecting to your home Wi-Fi (optional) . . . 9
- 2.4 Selecting the tester model . . . 9
- 2.5 LEDs on the board . . . 10
- 2.6 The web interface at a glance . . . 11

3 **Battery tester section** . . . 12
- 3.1 Measurements . . . 12
- 3.2 Battery . . . 13
- 3.3 Manual control . . . 15
- 3.4 Automatic control (test plans) . . . 16
- 3.5 Internal resistance (DCIR) . . . 18

4 **Administration section** . . . 22
- 4.1 Device information . . . 22
- 4.2 Test run management . . . 23
- 4.3 SD card . . . 24
- 4.4 Updates . . . 25
- 4.5 Settings . . . 26
- 4.6 Debug . . . 31

5 **Typical workflows** . . . 32
- 5.1 Measuring the capacity of a battery . . . 32
- 5.2 Measuring internal resistance at 50 % state of charge . . . 32
- 5.3 Backing up your data . . . 32

6 **Troubleshooting** . . . 33

Part II – Technical Reference . . . 34

7 **System architecture** . . . 35
- 7.1 Firmware structure . . . 35
- 7.2 Data flow . . . 35

8 **Hardware** . . . 36
- 8.1 Block diagram . . . 36
- 8.2 Circuit description . . . 36
- 8.3 Connectors, jumper and test points . . . 38
- 8.4 Bill of materials . . . 41

9 **Firmware: installation and configuration** . . . 43
- 9.1 Installing a release (no build environment) . . . 43
- 9.2 Building from source . . . 43
- 9.3 Firmware defaults and board revision 1 . . . 44
- 9.4 Partition table . . . 44
- 9.5 Status LED in firmware 1.4.72 . . . 45

10 **Communication with the tester** . . . 46
10.1 Serial interface . . . 46
10.2 Model detection . . . 46
10.3 Operating modes . . . 46
11 **Data storage** . . . 47
11.1 SQLite database on the SD card . . . 47
11.2 Schema versions . . . 47
11.3 MariaDB backup . . . 47
12 **Interfaces** . . . 49
12.1 HTTP API . . . 49
12.2 Home Assistant MQTT . . . 49
13 **Firmware updates and publishing** . . . 51
13.1 Update procedure . . . 51
13.2 Manifest format . . . 51
13.3 Publishing your own builds . . . 51
14 **Security and limitations** . . . 52
14.1 Security . . . 52
14.2 Known limitations (firmware 1.4.72) . . . 52
14.3 License . . . 52
Appendix A – Language files . . . 53
Appendix B – Glossary . . . 54

Part I – Operating Manual

This part explains how to set up and use EBCTester from day to day: connecting the tester, logging in, entering batteries, running manual tests, test plans and internal-resistance measurements, and reviewing and managing the recorded data. No programming or electronics knowledge is needed.

1 Introduction

1.1 What EBCTester does

ZKE (ZKETECH) EBC battery testers are popular, affordable instruments for charging and discharging batteries with constant current, constant power or constant voltage and for measuring capacity. Normally they are controlled by the manufacturer's Windows software, which means a PC has to stay connected for the whole test – often for days.

EBCTester replaces that PC. A small board with an ESP32-S3 microcontroller is connected to the tester's serial interface. It controls the tester, runs complete test plans on its own and records every measurement in a database on a microSD card. You operate it with any web browser – from a phone, a tablet or a PC – either through your home Wi-Fi or through the device's own Wi-Fi access point. You can close the browser at any time: the test continues and keeps being recorded.

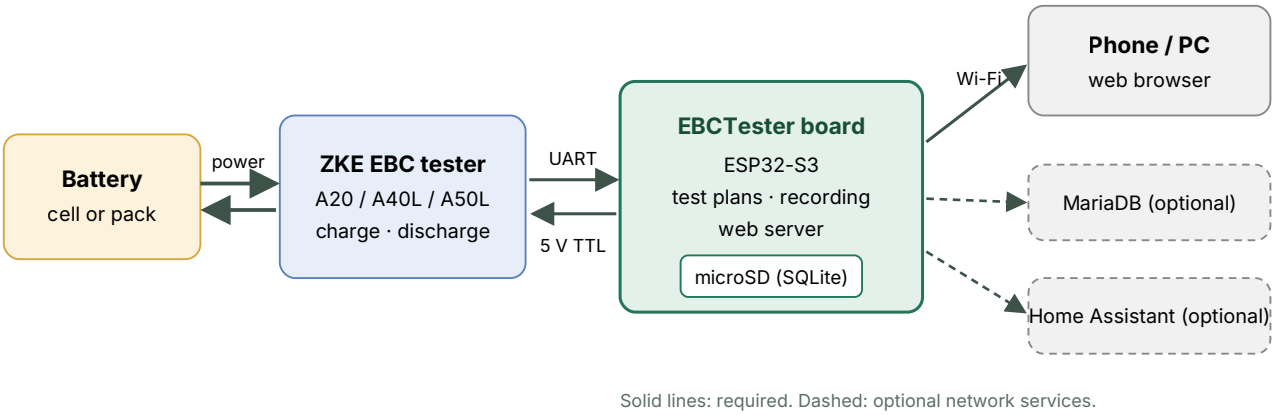


Figure 1: System overview

1.2 Main features

- **No PC needed during a test.** Manual operations and automatic test plans run on the ESP32-S3.
- **Offline recording.** All measurements go into an SQLite database on a FAT32 microSD card. Recording needs a ready SD card, not an Internet connection.
- **Browser interface.** Charts, battery records, test control and administration. Everything (including the chart library) is served by the device itself; no cloud service and no Internet access are required.
- **Test plans** with up to 10 steps: constant-current discharge, constant-power discharge, CC/CV charge, wait and repeat.
- **Internal resistance (DCIR)** by a load-pulse test with three separate results (by default after 1, 5 and 10 seconds), optionally with automatic preparation to a target state of charge.
- **Test-run management:** search, filter, rename and delete recorded runs.
- **Optional** backup to a MariaDB server (e.g. on a NAS), Home Assistant integration via MQTT and a live debug log.
- **Manual firmware updates** over Wi-Fi: new versions are announced, but only installed after you confirm.
- **Languages:** English and German built in; additional languages can be loaded from the SD card.

1.3 Supported testers

The firmware recognises a range of ZKE models by the identifier in their status messages. Only the models marked "enabled" can be controlled; all others are shown as read-only telemetry until their operation has been confirmed.

Model	Max. charge	Max. discharge	Max. power	Status
EBC-A20	18 V / 5 A	30 V / 20 A	85 W	enabled
EBC-A40L	5 V / 40 A	5 V / 40 A	200 W	enabled
EBC-A50L	5 V / 50 A	5 V / 50 A	250 W	enabled
EBC-A05(+), EBC-A10(+/H), EBC-A20+, EBD-M10, EBD-A10H, EBD-M03(+), EBD-M05, EBD-A100(L), EBC-A100L	recognised			read-only

These limits are enforced for every manual command, every plan step and every internal-resistance parameter. The A40L and A50L are low-voltage testers (max. 5 V): use them only with single lithium cells and settings that suit the cell. EBCTester has no battery-chemistry selection – the values you enter must be right for your battery.

> **Note**
> If you own one of the read-only models and can test it with EBCTester, please share your results via the GitHub project so the model can be enabled.

1.4 What you need

- An EBCTester board (or a compatible ESP32-S3 setup with 16 MB flash, see Part II) with installed firmware.
- A supported ZKE tester and a cable from the board's 6-pin connector J4 to the tester's serial (TTL) port.
- Power: the 19 V supply of the EBC-A20 (through J4) or a USB-C power supply.
- A 2.4 GHz antenna (the ESP32-S3-WROOM-1U module has no on-board antenna).
- A microSD card formatted as **FAT32** (exFAT is not supported). 4–32 GB is ideal.
- A CR2032 coin cell for the real-time clock (optional, see Part II).
- A phone, tablet or computer with a current web browser.

1.5 Safety information

> **Warning**
> Batteries store a lot of energy. Wrong settings, damaged cells or poor connections can cause overheating, fire or explosion. Lithium batteries in particular must never be charged above their maximum voltage or discharged below their minimum voltage.

- Enter charge voltages, currents and cut-off values that match the battery's data sheet. EBCTester only checks the limits of the *tester*, not those of your battery.
- The software does **not** replace hardware protection (BMS, fuse, temperature monitoring). Long tests are not automatically safe to leave unattended.
- If the serial connection is lost, EBCTester cannot guarantee that the tester stops. Check the tester locally after any connection error.

- Use cables and connectors rated for the test current, and keep the test setup on a non-flammable surface.
- The web interface uses unencrypted HTTP. Use it only in your own trusted network or through the device's protected access point – never expose it to the Internet.

2 Setting up

2.1 Connecting the hardware

1. **Antenna.** Mount the SMA socket of the antenna cable in the housing, plug its small u.FL plug onto the socket of the ESP32 module and screw on the antenna. Without an antenna Wi-Fi does not work.
2. **microSD card.** Insert a FAT32-formatted card. The card is never formatted automatically; a new, empty database is created on it at first use.
3. **Tester connection.** Connect J4 to the tester's serial interface (see the pin assignment in section 8.3). The board's TX line goes to the tester's RX line and vice versa. With the EBC-A20, J4 can also supply the board from the tester's 19 V adapter.
4. **Power.** Power the board either through J4 (19 V) or through USB-C. Both may be connected at the same time.

> **Caution**
> The pin order of the cable on the tester side differs between tester models (the EBC-A40L uses a different connector, and TX/RX have been reported swapped compared with the A20 cable). Check the pinout with a multimeter before connecting. The A40L and A50L are mains-powered and do not supply 19 V – power the board through USB-C in that case.

2.2 First start and login

On first start the device opens its own Wi-Fi access point. Its name is `BatteryTester-XXXXXX`, where *XXXXXX* is unique to the device. EBCTester uses two separate passwords:

Password	Used for	Initial value
Access-point password	Connecting to the Wi-Fi network `BatteryTester-XXXXXX`	Set when the firmware was built; otherwise generated randomly once and shown in the start-up log
Login password	Logging in to the web interface (user `admin`) and confirming every settings change	`ebctester`

To read a generated access-point password, connect the board to a PC by USB-C, open a serial terminal (e.g. the ESP-IDF monitor, PuTTY or the Arduino serial monitor) and press `RESET`. Note the password – it stays the same after restarts.

1. Connect your phone or PC to the Wi-Fi network `BatteryTester-XXXXXX` using the access-point password.
2. Open `http://192.168.4.1` in the browser.
3. Log in with user name `admin` and the login password – on a new device `ebctester`. The login page labels this field **Admin password**.
4. Change the login password right away under **Administration → Settings → Login Settings** (see Login Settings).

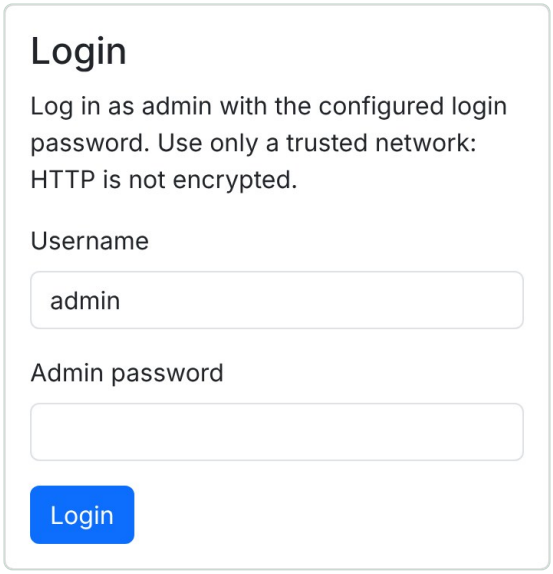


Figure 2: Login page

> **Caution**
> `ebctester` is the same on every device and is published in this manual. Change it on first use. Devices updated from firmware 1.4.71 or older also start with `ebctester` as long as no login password has been saved.

A login is valid for 24 hours or 7 days, depending on the setting. It is not extended by activity and ends when the device restarts or you click **Log out**. Running tests and recording are not affected by logging in or out. The login can also be switched off completely (see Login Settings).

2.3 Connecting to your home Wi-Fi (optional)

EBCTester works completely without home Wi-Fi. Connecting it to your network is still useful: you can reach it from every device in the house, the clock is synchronised from the Internet (needed for correct date/time stamps and for update checks) and optional services such as MariaDB and MQTT become possible.

1. Go to **Administration → Settings → Configure Wi-Fi**.
2. Enter the network name (SSID) and password and click **Save**. Confirm with the login password when asked.
3. The device connects; its new IP address is shown under **Device information → Wi-Fi IP**. Your router also lists it.

The protected access point remains active as a fallback, so you can always reach the device at `http://192.168.4.1`, even when the home network is out of range. Changing the Wi-Fi channel may briefly interrupt the access-point connection.

2.4 Selecting the tester model

Before the first connection you must tell EBCTester which tester you use – there is no default. Go to **Administration → Settings → EBC tester**, select the model and click **Save tester model**.

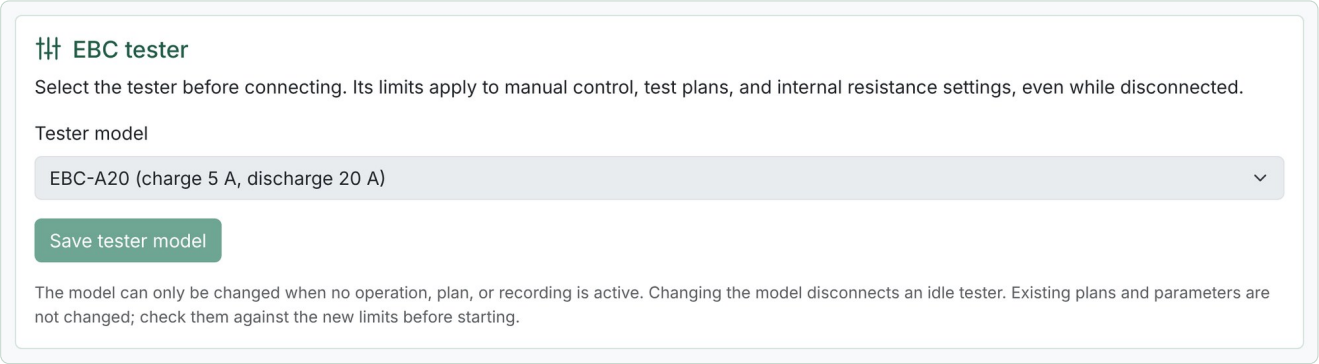


Figure 3: Selecting the tester model

The selected model and its limits apply immediately, even without a connected tester, so you can prepare plans and parameters in advance. The model can only be changed while no operation, plan or recording is active. If the connected tester reports a different supported model, EBCTester shows an error, disconnects and does not reconnect automatically until you correct the selection or click **Connect** again.

2.5 LEDs on the board

The EBCTester board has three LEDs. LED1 and LED2 are RGB LEDs; further LEDs can be connected to J5, e.g. for mounting in the enclosure lid.

LED	Display	Meaning
D6 – power	(yellow-)green, steady	The 3.3 V supply is OK. If D6 stays dark, check the 19 V or USB supply.
LED1 – status	off	Tester not connected
	green	Tester connected, no operation active
	blue	Manual operation running (Manual tab)
	red	Automatic operation running (test plan)
LED2 – RX	dim white	Tester not connected or no message for more than 5 s (grey in the web interface)
	dark green, steady	Tester connected, messages are arriving
	bright green flash	One flash for every valid message received from the tester
	orange	Communication error, e.g. invalid messages or a tester that does not match the selected model

LED2 uses the same colours as the RX indicator in the header of the web interface. The amber colour of the web interface means that the *browser* cannot reach the device; the board LED cannot show this and uses orange instead for errors in the communication with the tester.

> **Note**
> The LED functions of LED1 and LED2 described here are planned for a coming firmware version. Firmware 1.4.72 does not support them yet; it only uses LED1 to show the firmware-update state (see section 9.5). D6 works independently of the firmware.

2.6 The web interface at a glance

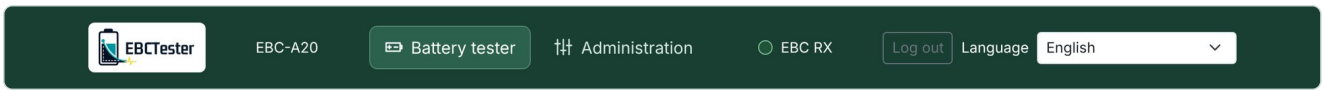


Figure 4: Header bar

The header bar is always visible. From left to right it contains:

Logo	Returns to the start page.
Model	The selected tester model (e.g. *EBC-A20*). It turns into a warning if the connected tester does not match or is unknown.
Battery tester / Administration	Switches between the two sections of the interface. Each section remembers its last open tab.
RX indicator	**Dark green** while the tester is connected, with a bright green flash for every valid message received from the tester. **Grey** = tester not connected, **amber** = the browser cannot reach the device. LED2 on the board shows the same (see section 2.5).
Log out	Ends the browser session. Hidden when browser login is switched off.
Language	English, Deutsch and any languages loaded from the SD card. The choice is stored in the browser; switching reloads the page (unsaved form entries are lost, a running test is not affected).

Section	Tabs
Battery tester	Measurements · Battery · Manual · Automatic · Internal resistance
Administration	Device information · Test run management · SD card · Updates · Settings · Debug (only when enabled)

Numeric input fields are checked as soon as you leave them. A red message below the field explains the allowed range; start and save buttons stay disabled until all fields are valid. When a start button is disabled for another reason (e.g. tester not connected), a yellow note below it tells you why.

> **Note**
> The screenshots in this manual were created with example data: an EBC-A20 testing a 50 Ah LiFePO4 cell. They show example settings, not recommendations for your battery.

3 Battery tester section

3.1 Measurements

The **Measurements** tab is the dashboard for recorded test runs. All data comes from the database on the SD card, so the page also works on the access point without Internet.

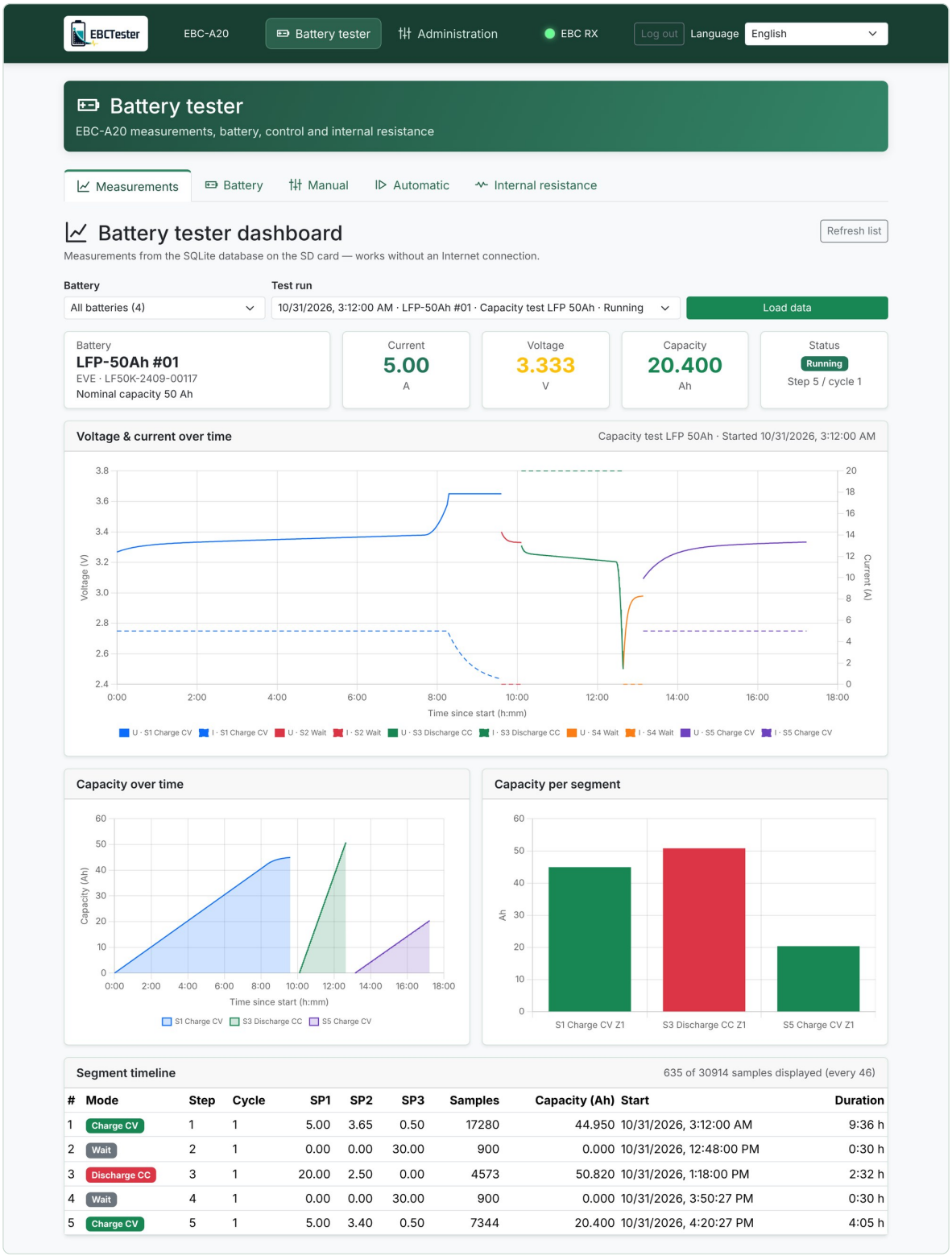


Figure 5: Measurements dashboard during a running capacity test

Selecting a test run

1. Optionally restrict the list with **Battery**.
2. Choose a run under **Test run**. Runs are listed newest first with date, battery, plan and status (or with the custom name given in test-run management).
3. Click **Load data**. **Refresh list** re-reads the list of runs, e.g. after a new test was started.

The newest run is loaded automatically when the page opens. A running test is refreshed every 15 seconds.

What is shown

- **Summary cards:** battery (label, manufacturer, serial number, nominal capacity), the latest current, voltage and capacity, and the run status with current step and cycle.
- **Voltage & current over time:** one colour per segment; voltage as solid line (left axis), current as dashed line (right axis). The time axis shows hours:minutes since the start of the run.
- **Capacity over time** for every charge or discharge segment, and **Capacity per segment** as a bar chart (green = charge, red = discharge).
- **Segment timeline:** one row per segment with mode, plan step, cycle, the three set-points (SP1–SP3), number of measurements, capacity, start time and duration.

A *segment* is a continuous section of a run with one operating mode – for example "charge CC/CV" or "wait". To keep the charts fast, at most 1500 points are drawn; the line above the timeline states how many of the stored points are shown ("every n-th"). Segment boundaries are always included. Hover over a chart (or tap it) to see the exact values.

Status	Meaning
Running	The test is still being recorded.
Completed	Finished normally (plan complete or cut-off condition reached).
Stopped	Stopped by the user.
Interrupted	Ended by a power loss, restart or lost connection.
Error	Ended because of an error (see the debug log for details).

3.2 Battery

Here you enter the battery that will be tested next. Its data is stored with every test run, so you always know which cell or pack a measurement belongs to.

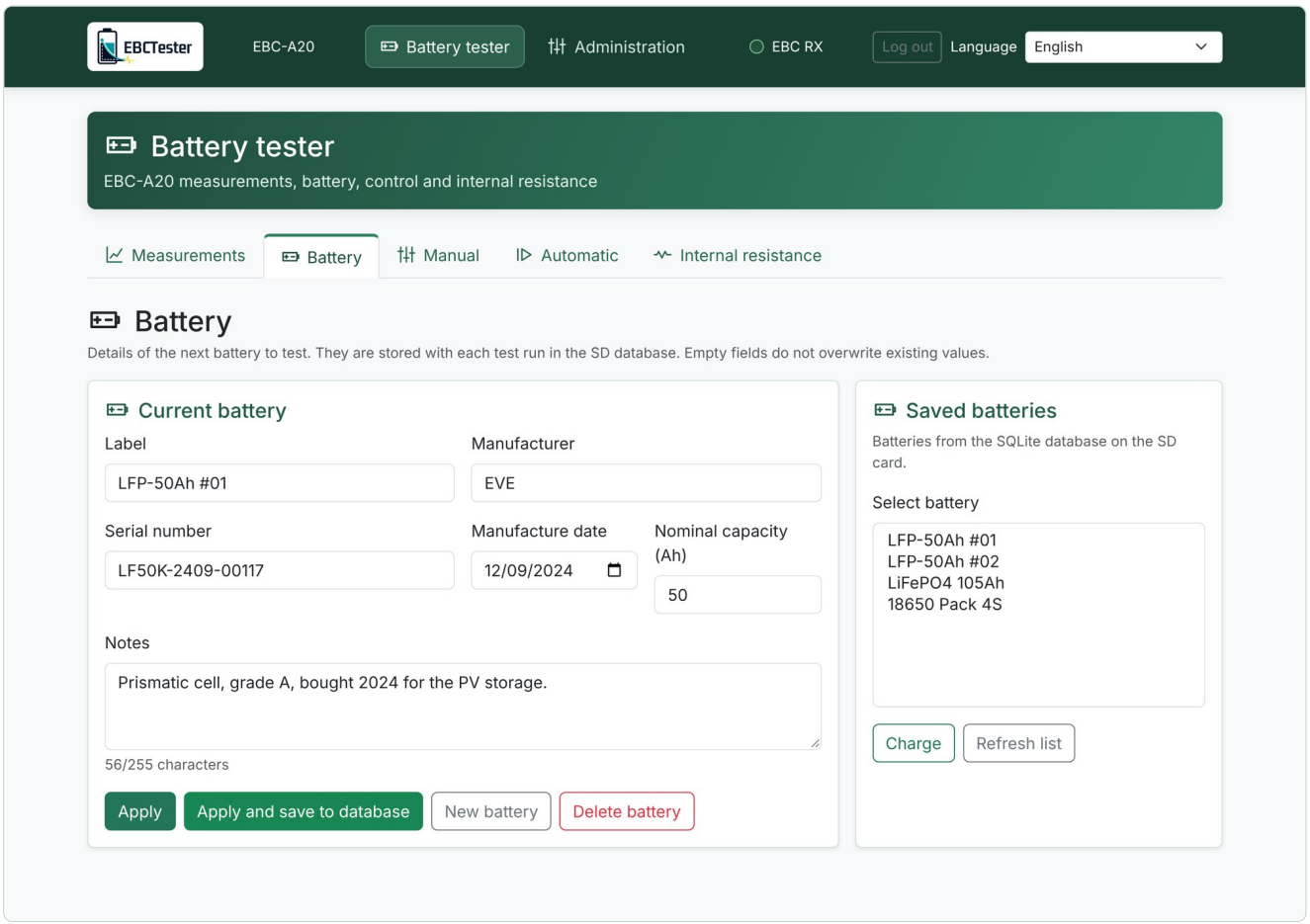


Figure 6: Battery tab

Field	Description
Label	Unique name of the battery, e.g. *LFP-50Ah #01*. Required for saving to the database.
Manufacturer, Serial number	Free text.
Manufacture date	Date picker.
Nominal capacity (Ah)	Rated capacity; shown in the dashboard for comparison.
Notes	Up to 255 characters.

Button	Function
Apply	Makes the entered battery the current battery (stored on the device). The next test run is recorded with it.
Apply and save to database	Additionally saves the battery in the SD database so it can be selected again later.
New battery	Clears the form for a new entry.
Delete battery	Deletes the saved battery shown in the form after confirmation and the login password. Not possible while it is used by recorded runs – delete those runs first in test-run management – or while a recording, plan or DCIR test is active. MariaDB backups are not changed.
Load button / Refresh list	Under **Saved batteries**: select a battery and click the button below the list (labelled **Charge** in firmware 1.4.72) or double-click the entry to load it into the form. **Refresh list** re-reads the database.

Empty fields do not overwrite values that are already stored.

3.3 Manual control

The **Manual** tab gives you direct control of the tester – comparable to the buttons on the tester itself, but with recording.

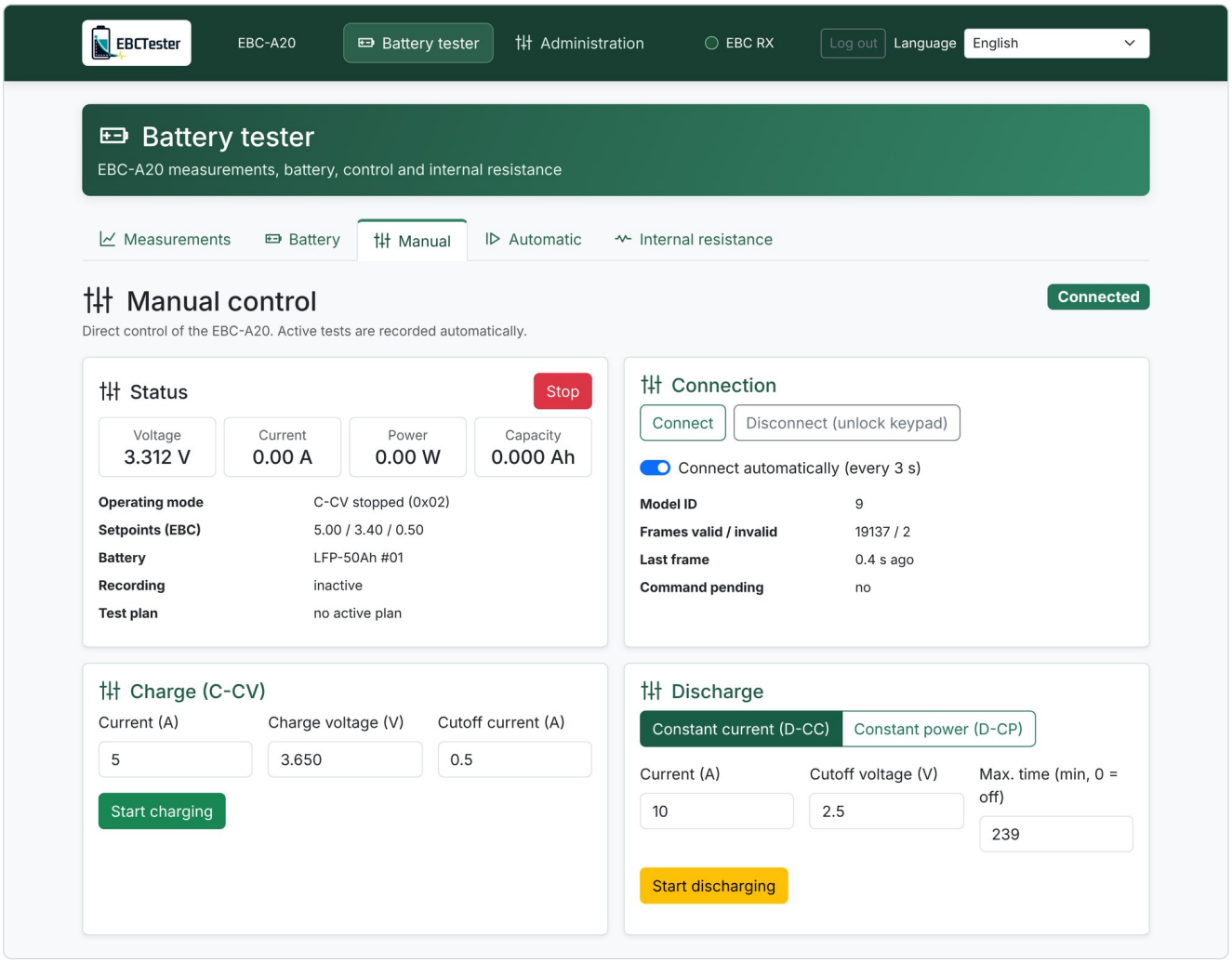


Figure 7: Manual control with the tester connected and idle

Status

Shows the live values from the tester (voltage, current, power, capacity), the operating mode with its code, the set-points currently active in the tester, the current battery, the recording state (number of stored measurements) and whether a test plan is active. The badge at the top right shows *Disconnected*, *Connected* or the active mode. **Stop** stops the current operation immediately; it is always available while connected.

Connection

- **Connect** starts communication with the tester. The tester's keypad is locked while connected.
- **Disconnect (unlock keypad)** ends the connection and unlocks the keypad.
- **Connect automatically (every 3 s)**: EBCTester tries to connect on its own, e.g. after switching the tester on.
- Diagnostic values: model ID reported by the tester, number of valid/invalid messages, age of the last message, and whether a command is waiting for confirmation.

Charge (C-CV)

Charges with constant current until the charge voltage is reached, then with constant voltage until the current falls below the cut-off current.

Field	Meaning	EBC-A20 range
Current (A)	Charge current in the CC phase	0.01 – 5 A
Charge voltage (V)	End-of-charge voltage (CV level)	up to 18 V
Cutoff current (A)	Charging ends when the current drops below this value	≤ charge current

Discharge

Select **Constant current (D-CC)** or **Constant power (D-CP)**.

Field	Meaning	EBC-A20 range
Current (A) / Power (W)	Discharge current (D-CC) or power (D-CP)	0.01 – 20 A / 1 – 85 W
Cutoff voltage (V)	Discharging ends at this battery voltage	up to 30 V
Max. time (min, 0 = off)	Additional time limit	0 – 239 min

In D-CP mode the power must also be achievable within the maximum current at the cut-off voltage (power ≤ max. current × cut-off voltage). For the A40L/A50L the ranges of the respective model apply.

After **Start charging** or **Start discharging**, EBCTester waits up to 10 seconds for the tester to confirm the mode; otherwise it reports an error and sends Stop. Every manual operation is recorded automatically as a test run.

> **Note**
> Start buttons are disabled while a command is pending, while another operation or a test plan is running (including its wait steps) and while no tester is connected. The reason is shown below the button.

3.4 Automatic control (test plans)

A test plan is a sequence of up to **10 steps** that EBCTester executes on its own. The plan runs on the ESP32 and continues without Wi-Fi and without an open browser as long as the device is powered.

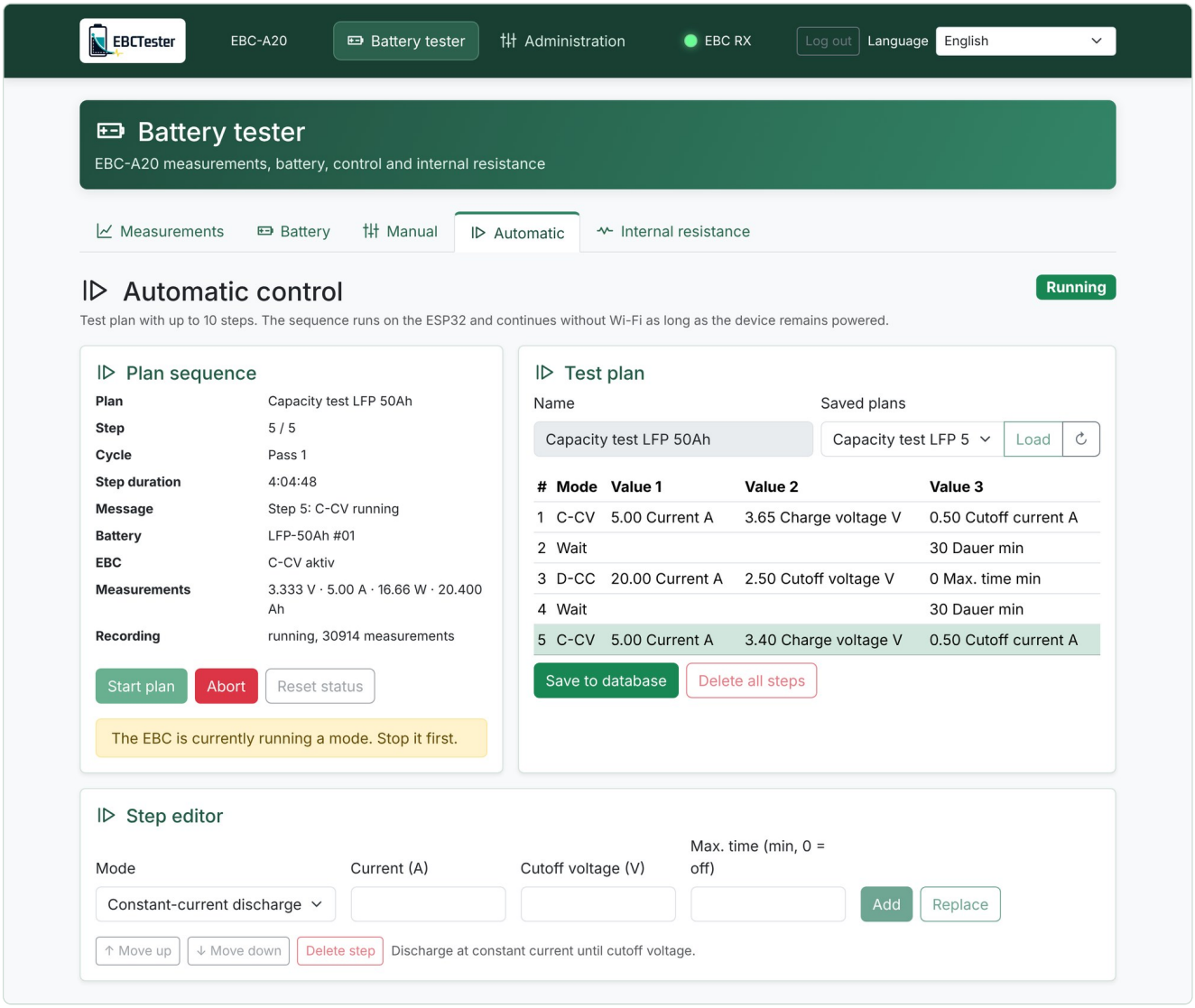


Figure 8: Automatic control: running plan (left) and plan editor (right)

Step types

Mode	Value 1	Value 2	Value 3	Function
D-CC	Current (A)	Cutoff voltage (V)	Max. time (min, 0 = off)	Discharge at constant current until the cut-off voltage.
D-CP	Power (W)	Cutoff voltage (V)	Max. time (min, 0 = off)	Discharge at constant power until the cut-off voltage.
C-CV	Current (A)	Charge voltage (V)	Cutoff current (A)	Charge CC/CV until the cut-off current.
Wait	–	–	Duration (1 – 239 min)	Pause, e.g. to let the cell recover.
Cycle	–	–	Total passes (1 – 239)	Returns to step 1 until the number of passes is reached. Not allowed as first step and only once per plan.

Creating and editing a plan

1. Enter a **Name** for the plan.
2. In the **Step editor**, choose the mode, fill in the values and click **Add**. The step is appended – or inserted after the selected row.

3. Click a row in the table to select it. You can then **Replace** it with the editor values, move it (**↑ Move up**, **↓ Move down**) or **Delete step**. Click the row again to deselect it.
4. **Save to database** stores the plan under its name on the SD card. **Delete all steps** clears the table.

Every change is saved on the device immediately and checked against the limits of the selected tester. Saved plans can be loaded again with the **Saved plans** list and **Load**; ↻ refreshes the list.

Example: capacity test of a LiFePO4 cell

#	Mode	Values	Purpose
1	C-CV	5.00 A · 3.65 V · 0.50 A	Charge fully
2	Wait	30 min	Let the cell settle
3	D-CC	20.00 A · 2.50 V · 0 (no time limit)	Measure the capacity
4	Wait	30 min	Let the cell recover
5	C-CV	5.00 A · 3.40 V · 0.50 A	Charge to a storage level

Example values only – use the values from your cell's data sheet.

Running a plan

Connect the tester, check the current battery and click **Start plan**. The **Plan sequence** card shows the plan name, current step, cycle, time in the step, status message, battery, tester mode, live values and the number of recorded measurements. The active step is highlighted green in the table.

- **Abort** stops the plan and the tester after confirmation.
- **Reset status** returns a completed, aborted or interrupted plan to "Ready" so it can be started again.

> **Caution**
> After a power failure or restart, a running plan is **not** resumed automatically. It is marked *Interrupted*. Check the battery, then click **Reset status** and start again if required.

3.5 Internal resistance (DCIR)

This tab measures the DC internal resistance of a battery with a load pulse. The battery rests without load, then a defined discharge current is switched on. From the voltage drop the resistance is calculated three times – by default 1, 5 and 10 seconds after switching on:

$$R = (U_1 - U_2) / I_2$$

U_1 is the average rest voltage at the end of the rest phase, U_2 and I_2 are the voltage and the *measured* current at the sampling time. Short times mainly reflect the ohmic resistance (contacts, electrolyte); longer times include polarisation effects.

Measurement parameters

Preparation

Automatic preparation and measurement

Automatic preparation

Usable capacity (Ah)	Charge current (A)
50	5
Charge voltage (V)	Charge cutoff current (A)
3.6	0.5
Preparation discharge current (A)	Maximum charging time (s)
5	43200
Maximum preparation discharge time (s)	
43200	

Rest period and load pulse

Prepared / target state of charge (%)	Rest duration (s)
50	1800
U1 window at end of rest (s)	Maximum idle current (A)
3	0
Load-pulse current (A)	Load-pulse duration (s)
20	12
R1 sample time (s)	R2 sample time (s)
1	5
R3 sample time (s)	
10	

Protection limits and measurement quality

Minimum / cutoff voltage (V)	Maximum battery voltage (V)
2.5	3.65
Maximum power (W)	Maximum UART sampling gap (s)
85	3
Maximum sample delay (s)	Minimum voltage drop (V)
1.5	0.001

☐ Charging parameters and current/voltage/power limits checked for this battery.

Start measurement | Save parameters | Abort / stop

Confirm that the charging parameters and current/voltage/power limits have been checked.

Figure 9: Measurement parameters

Sequence

Battery	LFP-50Ah #01
Phase	Completed
Phase duration	7412.6 s
Removed / target	25.0412 / 25.0000 Ah
U_1 / I_1	3.2934 V / 0.000 A
Live U / I	3.312 V / 0.000 A
Onset sampling gap	0.250 s
Message	Measurement complete
Test run	20261025T140210Z-05

Results

Requested / actual (s)	U_2 (V)	I_2 (A)	R (mΩ)	Measurement time
1.0 / 1.020	3.2581	19.980	1.420	10/26/2026, 3:12:01 AM
5.0 / 5.040	3.2456	19.980	2.050	10/26/2026, 3:12:05 AM
10.0 / 10.010	3.2361	19.980	2.530	10/26/2026, 3:12:10 AM

$R = (U_1 - U_2) / I_2$. Raw values without 20-frame averaging. Time zero: start command after the last idle frame. Actual load onset is between that frame and the first loaded frame. UART sampling rate and resolution particularly limit the 1-second result. Late or implausible results are not reported as resistance.

Figure 10: Sequence and results of a completed test

Preparation methods

- **State of charge already prepared:** the battery is already at the desired state of charge. The test starts with the rest phase.
- **Automatic preparation and measurement:** the battery is first fully charged (C-CV), then the calculated amount of charge is removed: *usable capacity × (100 − target SOC) / 100*. The removed charge is integrated from the measured current. The target SOC is therefore an estimate based on the entered capacity, not a measured value. A target of 100 % skips the discharge.

Parameters

Group / parameter	Meaning	Default
Automatic preparation (only with automatic preparation)		
Usable capacity (Ah)	Capacity used to calculate the charge to remove	20
Charge current / voltage / cutoff current	Full charge before the measurement	0.5 A / 3.45 V / 0.1 A
Preparation discharge current (A)	Current used to reach the target SOC	0.5
Max. charge / preparation time (s)	Time-outs for both phases	86400
Rest and load pulse		
Prepared / target SOC (%)	Target state of charge (automatic) or information (prepared)	50
Rest time (s)	Rest without load before the pulse	60
U1 window at end of rest (s)	Averaging window for U_1	3
Max. idle current (A)	Current that still counts as "no load"	0.02
Pulse current (A) / duration (s)	Load pulse	0.5 A / 12 s
Sample times R1 / R2 / R3 (s)	Times of the three results after switching on	1 / 5 / 10
Protection limits and measurement quality		
Minimum / cutoff voltage (V)	Test stops below this voltage	2.5
Maximum battery voltage (V)	Test stops above this voltage	3.6
Maximum power (W)	Upper limit for current × max. voltage	50
Max. UART gap (s)	Longest permitted interval between tester messages	3
Max. sample delay (s)	Timing tolerance of the samples	1.5
Min. voltage drop (V)	Smaller drops are rejected as not credible	0.001

Consistency rules: sample times must increase; the pulse must last at least as long as the last sample time plus the timing tolerance; the rest time must be at least the U1 window; cut-off and charge voltage must be below the maximum voltage; and current × maximum voltage must not exceed the maximum power. In addition, all values must be within the limits of the selected tester.

Running the measurement

1. Connect the battery with **four-wire sensing** (separate sense leads) – otherwise the cable resistance is included in the result.
2. Check all parameters and click **Save parameters**.
3. Tick the confirmation box ("Charging parameters and current/voltage/power limits checked for this battery"). Changing any parameter clears this box again.

4. Click **Start measurement**. The SD database must be ready.
5. Follow the progress in the **Sequence** card; **Abort / stop** ends the test at any time.

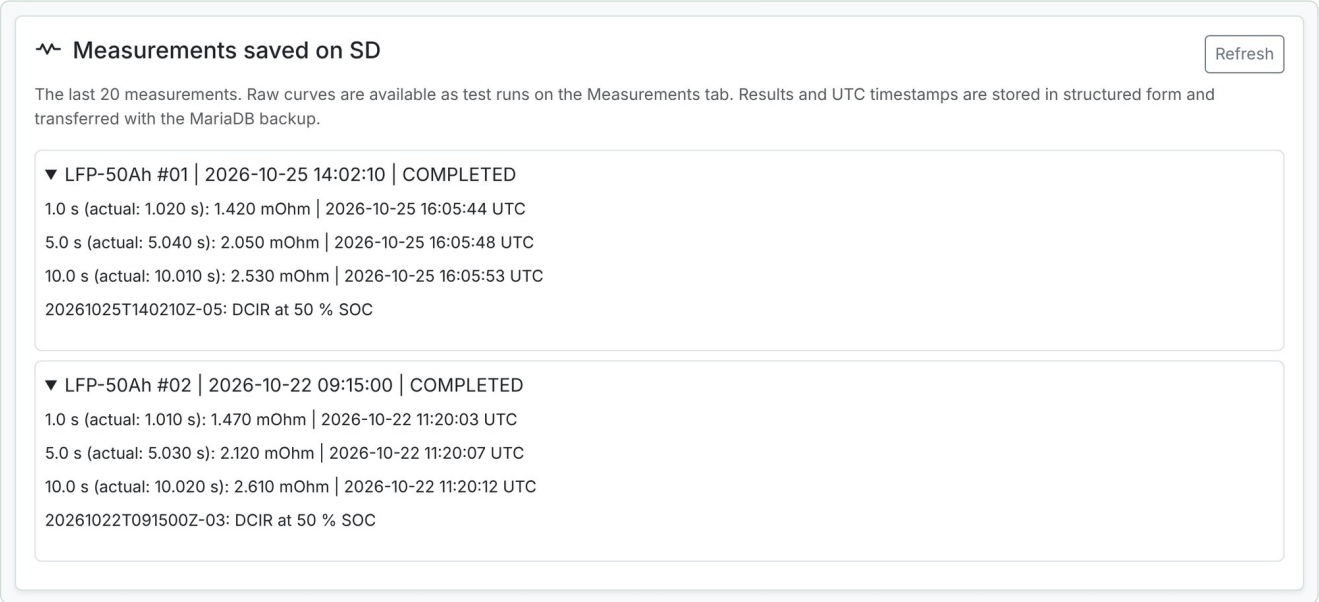


Figure 11: Measurements saved on SD

The **Results** table lists for each sampling time the requested and actual time, U_2, I_2, the resistance in mΩ and the time stamp. Late or missing samples, too small or negative voltage drops or missing current produce an "invalid" result instead of a wrong number. *Onset sampling gap* shows the uncertainty of the switch-on moment caused by the interval between tester messages – this mainly limits the accuracy of the 1-second value.

Every DCIR test is stored as a normal test run (raw curve in **Measurements**) plus structured results. **Measurements saved on SD** lists the last 20 DCIR tests. Temperature is not measured and is reported as unknown.

> **Note**
> During a DCIR test all other starts, connect/disconnect and battery changes are blocked. Stop and Abort always remain available. A DCIR test is never resumed after a restart.

4 Administration section

4.1 Device information

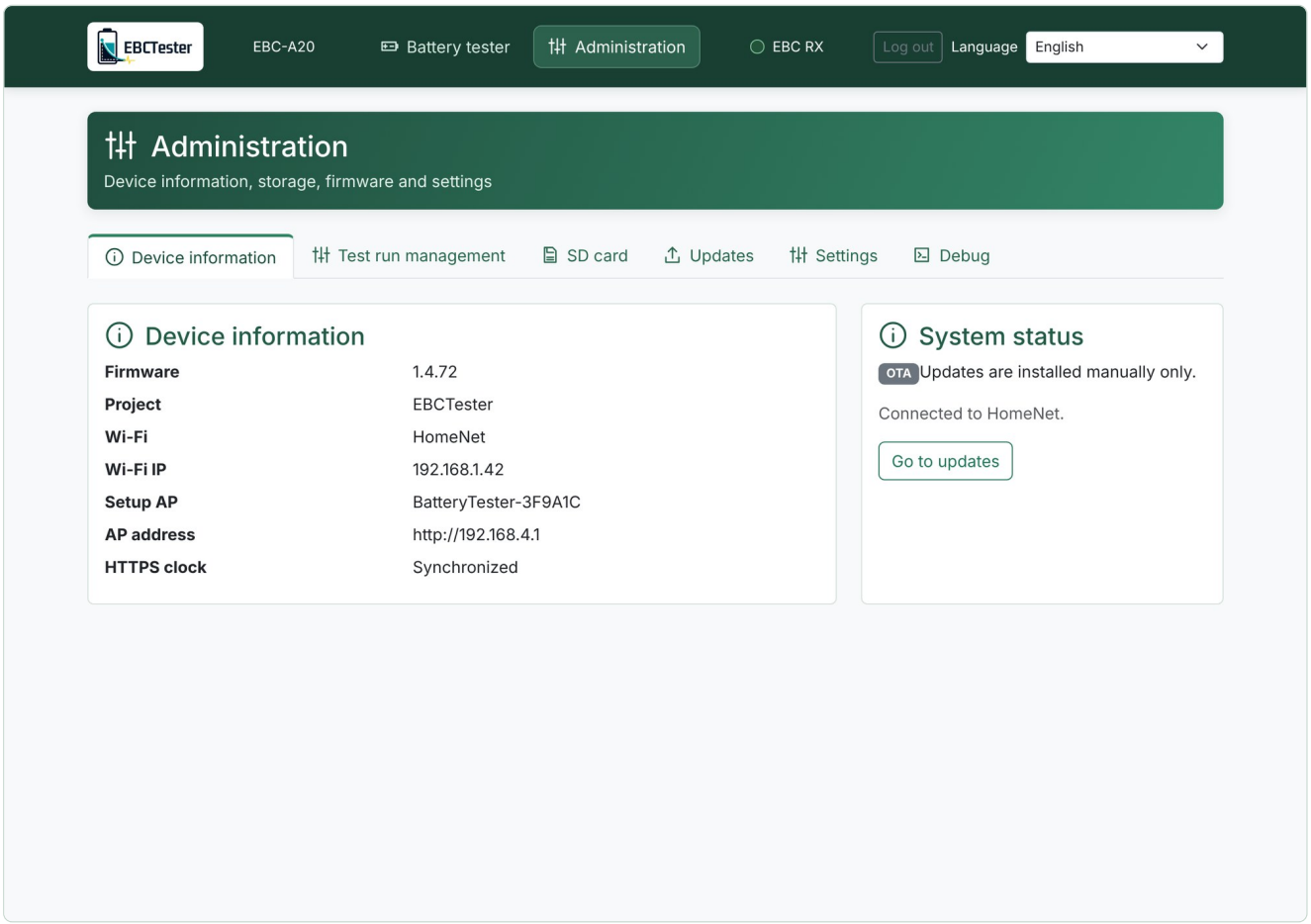


Figure 12: Device information

Shows the installed firmware version, the project name, the connected Wi-Fi network and IP address, the name of the setup access point (`BatteryTester-XXXXXX`), its address `http://192.168.4.1` , and whether the clock has been synchronised ("HTTPS clock"). **System status** shows the network message and leads to the update page.

4.2 Test run management

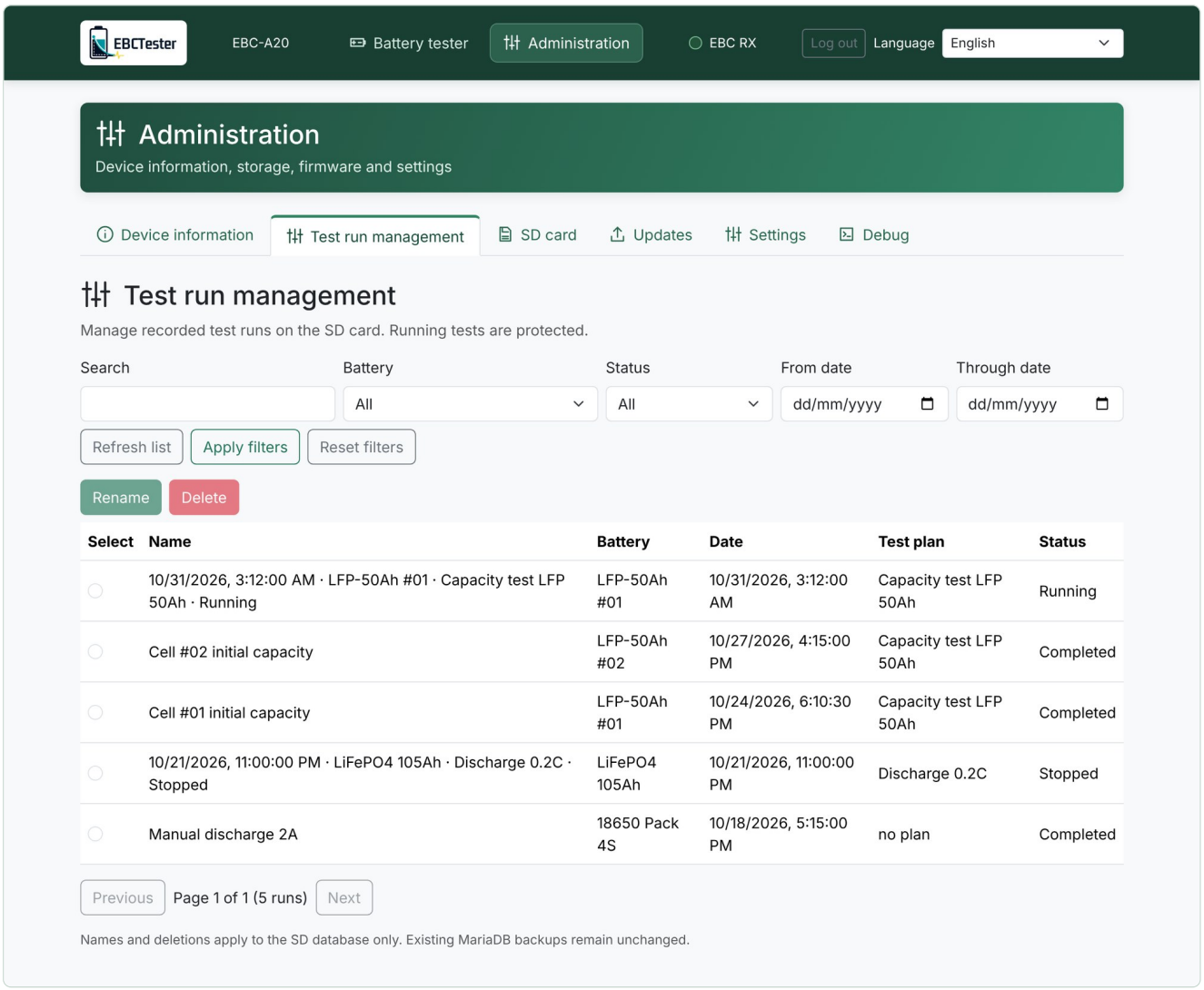


Figure 13: Test run management

This table lists *all* runs in the SD database, 50 per page, newest first.

- **Filter:** free-text **Search** (name, battery, plan, date, status or run ID), **Battery**, **Status** and a date range (**From date** / **Through date**, the end date includes the whole day). Click **Apply filters**; **Reset filters** clears them.
- **Rename:** select one run with the radio button and click **Rename**. Up to 100 characters; an empty name restores the automatic name. The name is also used in the dashboard.
- **Delete:** select a run and click **Delete**. After confirmation the run is permanently removed together with its measurements, segments and DCIR results. Batteries and plans are kept.

Running runs are protected and cannot be renamed or deleted. Renaming and deleting require the login password and only affect the SD database – existing MariaDB backups are not changed, and custom names are not transferred to MariaDB.

> **Caution**
> Deleting cannot be undone. Make a backup (MariaDB synchronisation or a copy of the database file) before deleting important runs, and do not delete runs while a MariaDB synchronisation is running.

4.3 SD card

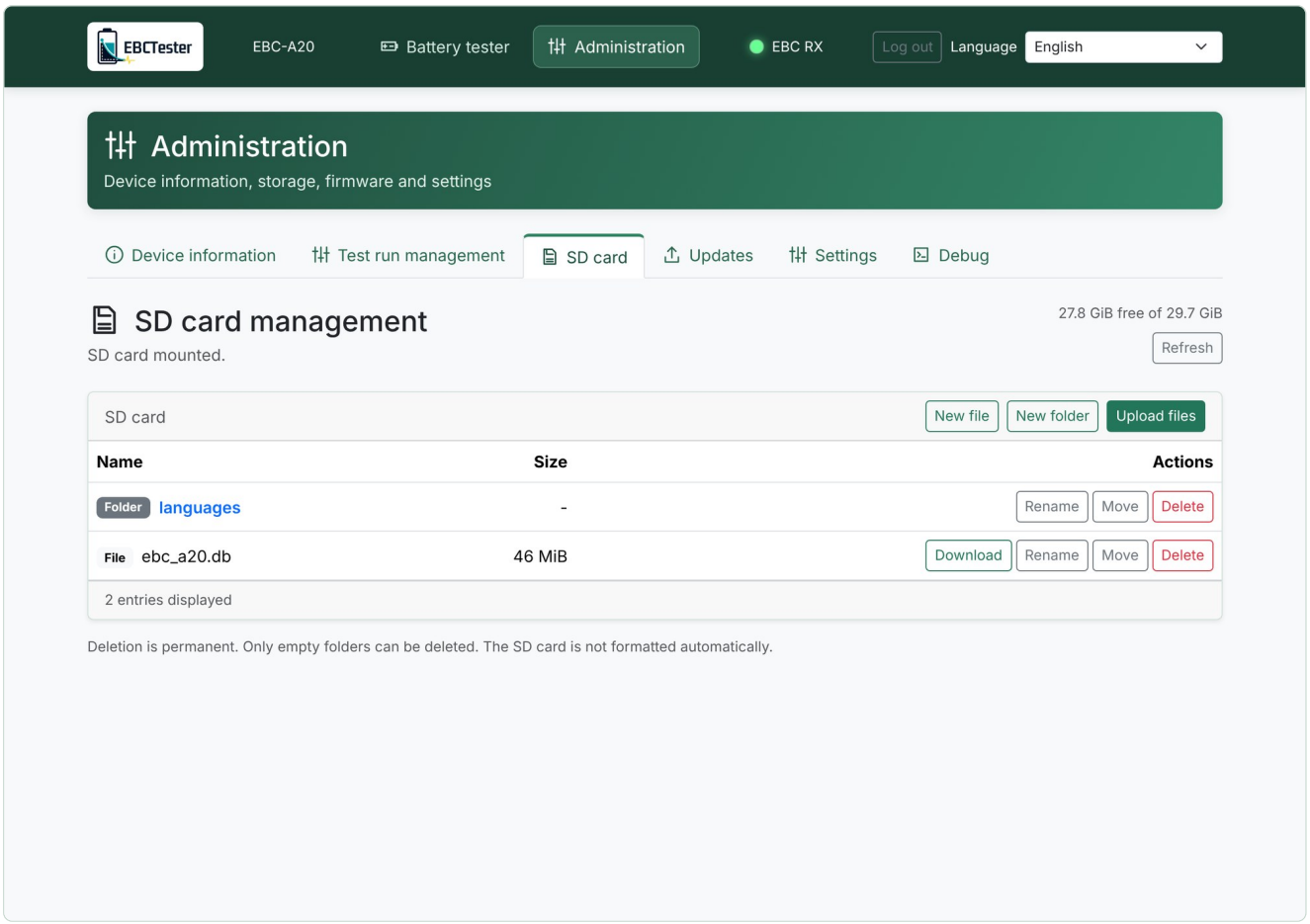


Figure 14: SD card management

A simple file manager for the microSD card – no need to remove it from the device. The header shows whether a card is mounted and how much space is free.

- Click a folder name to open it; use the breadcrumb path to go back.
- **New file**, **New folder** and **Upload files** (several files possible, with progress bar).
- For each entry: **Download** (files only), **Rename**, **Move** and **Delete**.

Deleting is permanent; only empty folders can be deleted. The card is never formatted automatically. The measurement database is the file `ebc_a20.db` in the root folder; download it regularly as a backup.

> **Caution**
> Do not rename, move or delete the database file while the device is in use. Download it only when no test is being recorded to get a consistent copy.

4.4 Updates

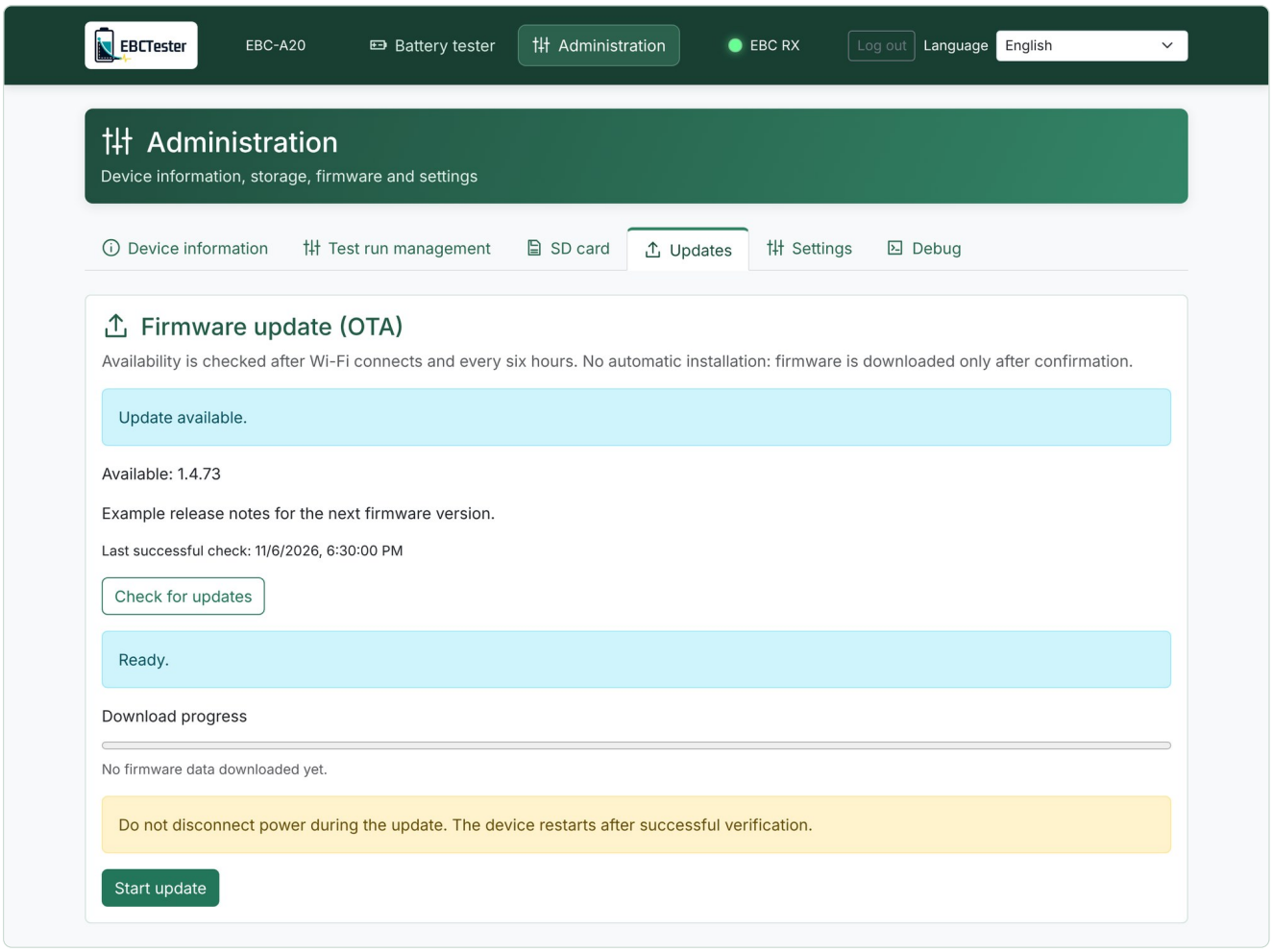


Figure 15: Firmware update

When the device is connected to the Internet it checks for new firmware after connecting, every six hours and once each time you open the web interface. Only a small description file is downloaded – **firmware is never installed automatically.**

1. If an update is available, the new version and its release notes are shown. **Check for updates** repeats the check manually.
2. Make sure no test is running, then click **Start update** and confirm the version.
3. The firmware is downloaded into the inactive memory area and checked (size and SHA-256 checksum). Progress is shown.
4. After successful verification the device restarts after three seconds. The new firmware is only accepted permanently if it runs stable for 10 seconds; otherwise the previous version is restored automatically.

> **Warning**
> Do not disconnect the power during an update, and never update during a battery test.

4.5 Settings

The **Settings** tab collects all device settings. When you save a setting you are asked for the current **login password** (labelled **Admin password** in the dialog) as an extra protection – also when browser login is switched off. Cancelling or entering a wrong password does not save anything. The access-point password is not used here.

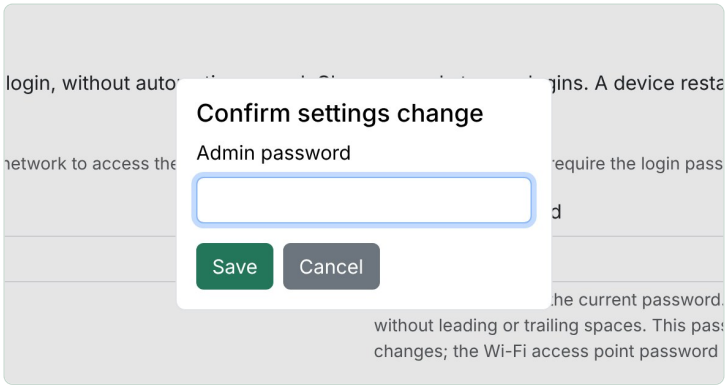


Figure 16: Password confirmation when saving settings

Login Settings

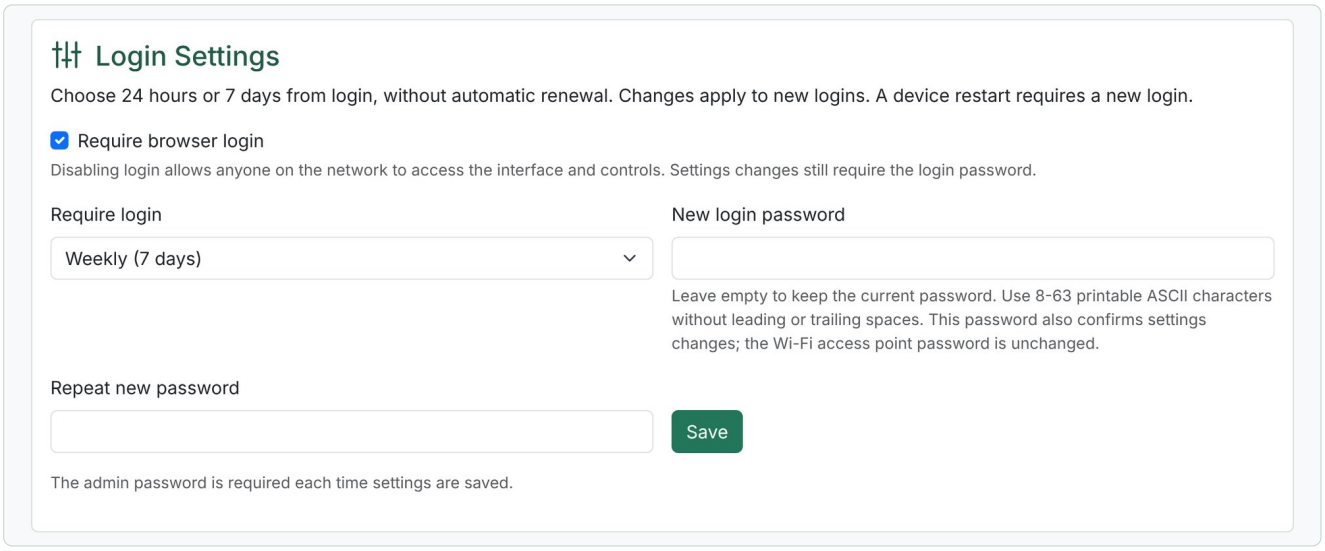


Figure 17: Login Settings

Setting	Description
Require browser login	On by default. When switched off, the web interface opens without logging in and the **Log out** button disappears.
Require login	**Daily (24 hours)** (default) or **Weekly (7 days)**. The duration counts from the login, is not extended by activity and applies to new logins. Only available while browser login is on.
New login password / Repeat new password	Enter the new password twice. 8–63 printable ASCII characters (letters, digits and common symbols, no umlauts), no spaces at the beginning or end. Leave both fields empty to keep the current password.

Click **Save** and confirm with the *current* login password – also when you are changing the password itself. The new password is stored permanently on the device and survives restarts and firmware updates. The access-point password is not changed.

Changing the password or switching browser login on or off ends all existing sessions; the page reloads automatically. Afterwards log in with the new password.

> **Warning**
> With browser login switched off, **everyone in the network** can view all data and operate the tester – including starting and stopping tests. Use this only in a network you trust completely. Settings changes still require the login password.

Configure Wi-Fi

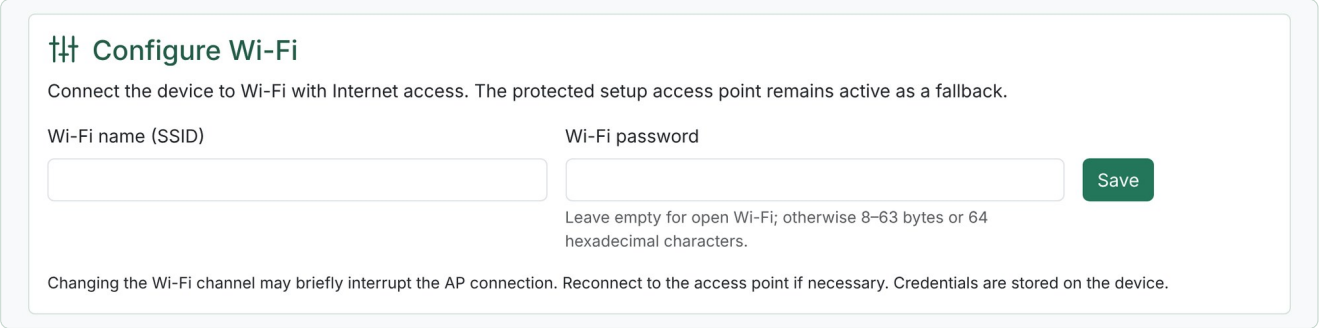


Figure 18: Wi-Fi settings

Network name (SSID) and password of your home network. Leave the password empty for an open network; otherwise 8–63 characters or 64 hexadecimal characters. The credentials are stored on the device.

EBC tester

Selection of the tester model – see Selecting the tester model.

Home Assistant MQTT

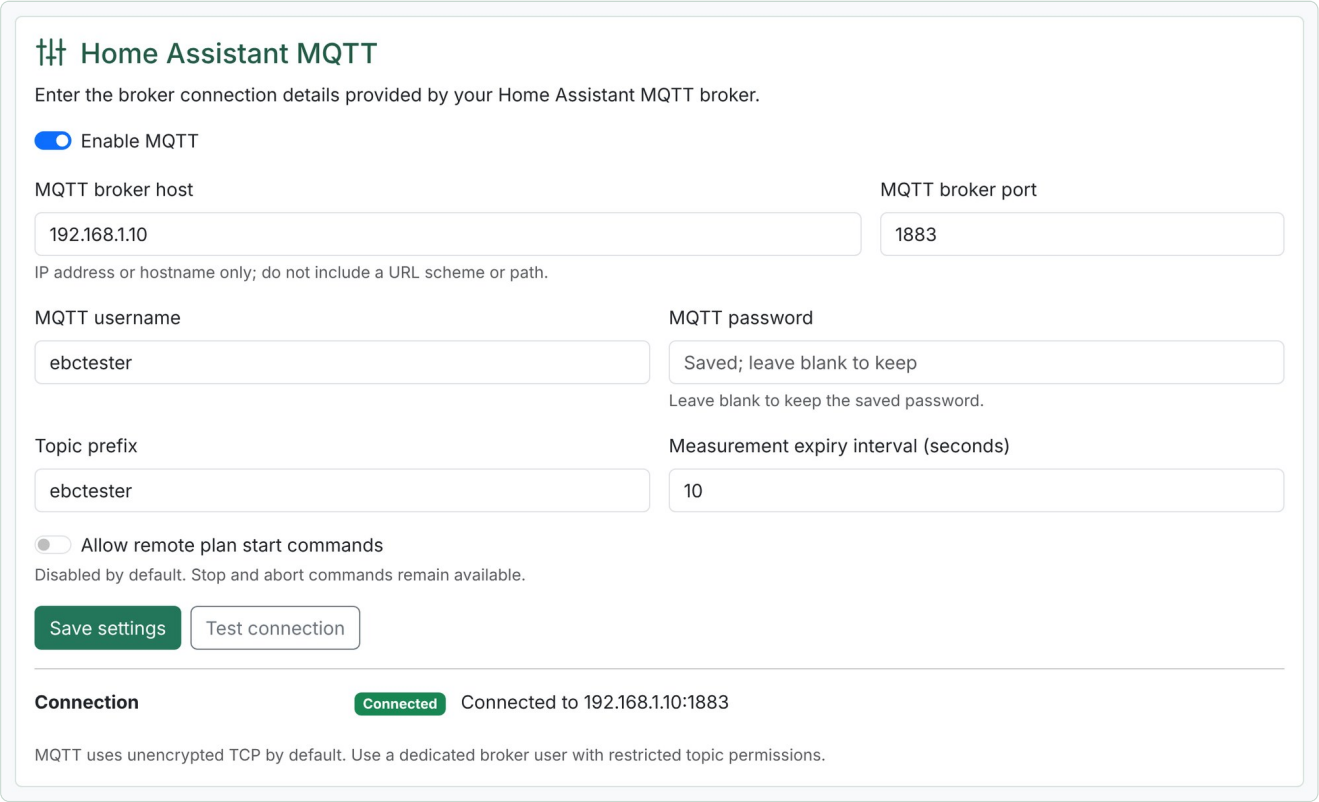


Figure 19: Home Assistant MQTT (only present if built into the firmware)

Optional connection to an MQTT broker, typically the one used by Home Assistant. The device then appears automatically in Home Assistant (MQTT discovery) and publishes voltage, current and capacity of every recorded measurement.

Enable MQTT	Switches the client on or off.
Broker host / port	IP address or host name (without `mqtt://`), port usually 1883.
User name / password	Broker account. Leave the password empty to keep the saved one.
Topic prefix	Start of all topics, e.g. `ebctester`.
Measurement expiry interval (s)	Interval between recorded measurements; values become "unavailable" after three intervals without new data.
Allow remote plan start commands	Off by default. Only when enabled can a plan be started via MQTT. Stop and Abort always work.

Save settings applies the settings immediately, **Test connection** reconnects with the saved settings. The connection state is shown below. MQTT is unencrypted – use a dedicated broker user with restricted rights. Details are given in section 12.2.

MariaDB backup

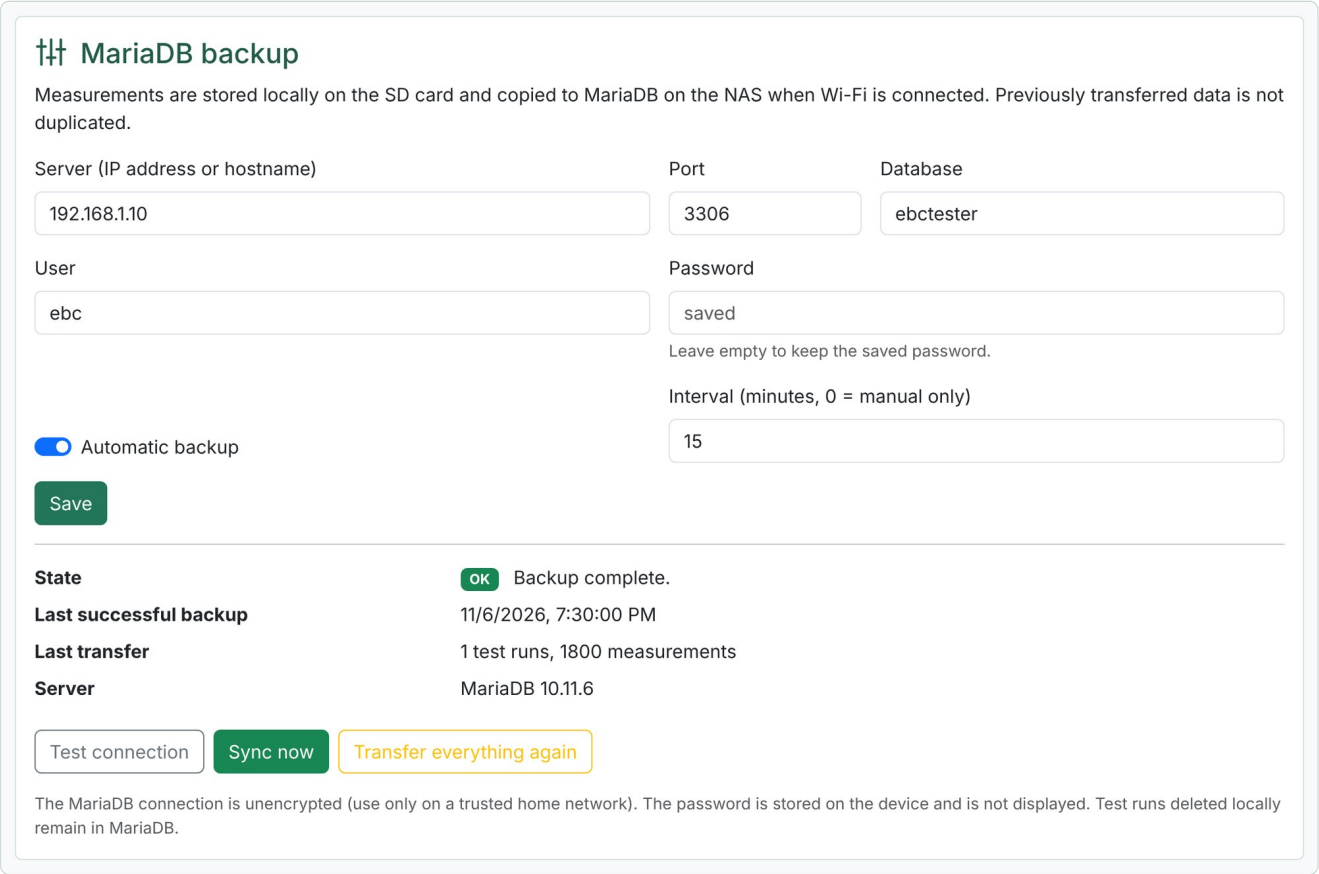


Figure 20: MariaDB backup

Copies the recorded test runs to a MariaDB database, for example on a NAS. Only new data is transferred; repeated or interrupted synchronisations do not create duplicates.

- Enter server, port (3306), database, user and password; choose **Automatic backup** and an interval in minutes (0 = manual only). Save.
- **Test connection** checks access, **Sync now** starts a transfer immediately, **Transfer everything again** re-sends all data (no duplicates are created).
- The status area shows state, last successful backup, number of runs and measurements transferred, and the server version.

Automatic backup only runs with Wi-Fi, a ready SD card and a saved password. Runs deleted on the device remain in MariaDB. The database tables must already exist on the server – see section 11.3. The connection is unencrypted: use it only in your home network.

Debug log

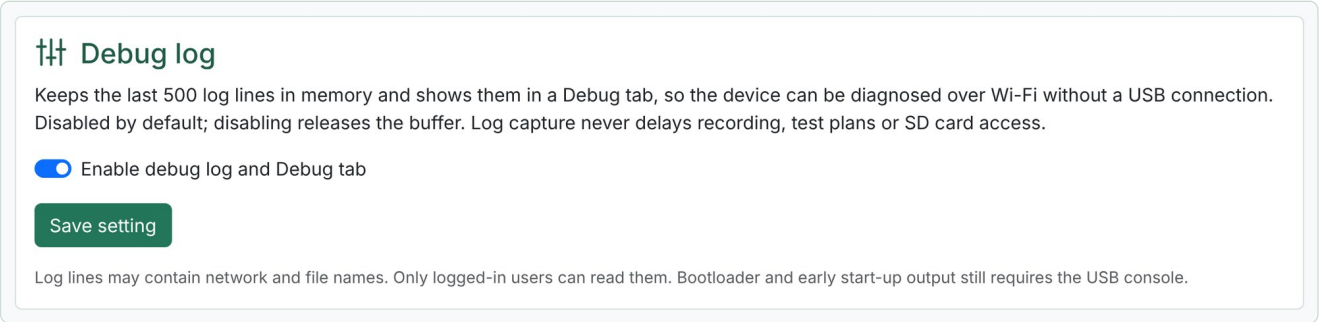


Figure 21: Debug log setting

When enabled, the device keeps the last 500 log lines in memory and shows them in the **Debug** tab. Disabled by default; disabling frees the memory. Logging never delays recording, plans or SD access.

Language



Figure 22: Language

English and German are built in. Additional languages can be added as JSON files in the folder `/languages` on the SD card (see Appendix A). **Reload languages** reads the folder again; then select the language in the header.

4.6 Debug

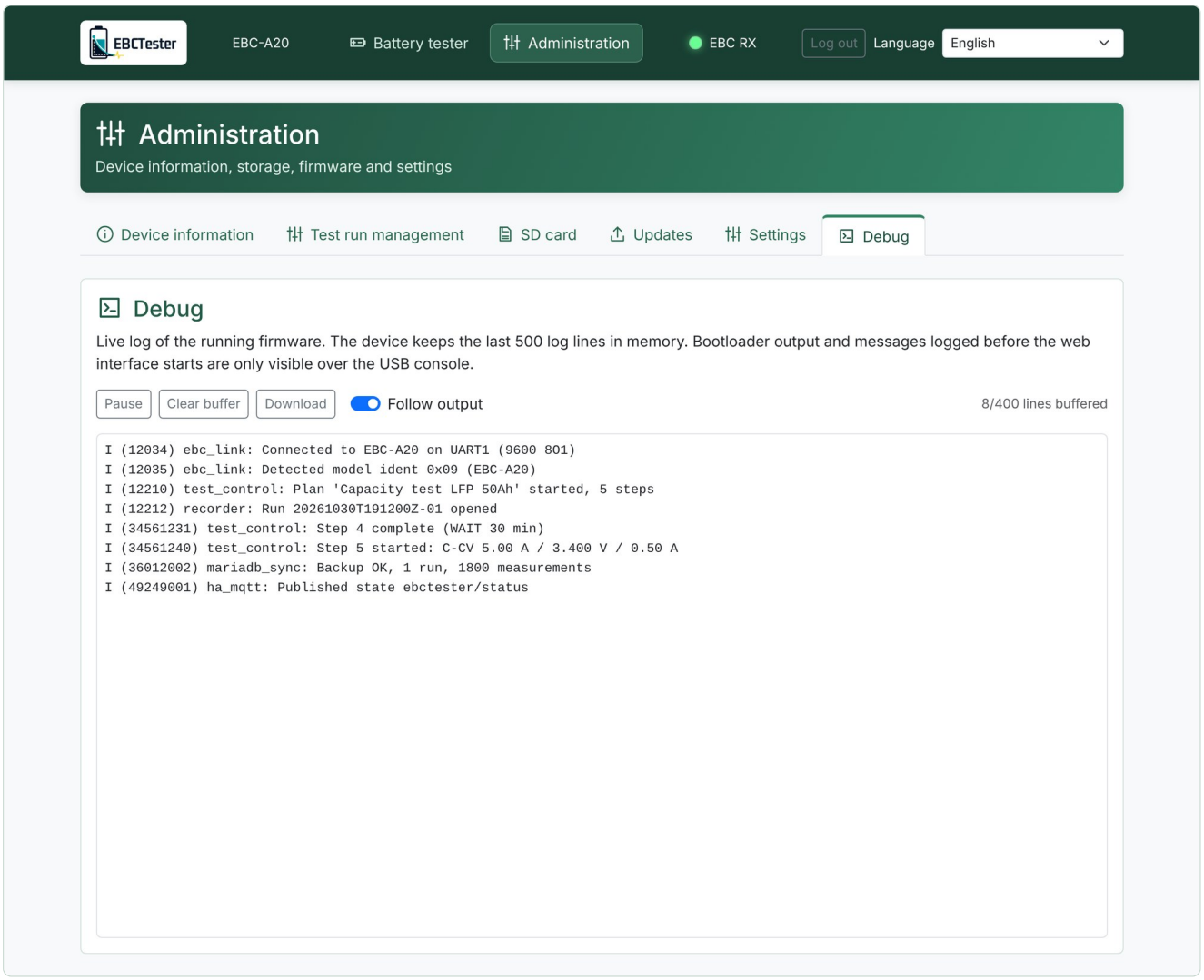


Figure 23: Debug tab

Live log of the running firmware – useful when something does not work and no USB cable is at hand. **Pause** freezes the display, **Clear buffer** empties it, **Download** saves the lines as a text file, and **Follow output** keeps the view at the newest line. Log lines may contain network and file names, but never passwords. Messages from the very beginning of the start-up are only visible on the USB console.

5 Typical workflows

5.1 Measuring the capacity of a battery

1. **Battery:** enter the battery data and click **Apply and save to database** (or load a saved battery and click **Apply**).
2. **Plan:** in **Automatic**, load or create a plan: charge (C-CV) – wait – discharge (D-CC) – wait – optionally charge to storage level.
3. **Connect:** in **Manual**, click **Connect** (or enable automatic connection). The RX indicator in the header blinks.
4. **Start:** in **Automatic**, click **Start plan**. You can now close the browser.
5. **Follow:** open **Measurements** at any time. The capacity of the discharge segment in the segment timeline is the measured capacity.
6. **Finish:** when the plan is complete, the run status changes to *Completed*. Give the run a meaningful name in **Test run management** if you like.

5.2 Measuring internal resistance at 50 % state of charge

1. Apply the battery as above and connect with four-wire sensing.
2. In **Internal resistance**, select **Automatic preparation and measurement**, enter the usable capacity (ideally the result of a capacity test), the charge parameters and *Target SOC = 50 %*.
3. Set the pulse current (e.g. 0.2 C to 1 C, within the battery's limits), check the protection limits, save, tick the confirmation and start.
4. After completion compare the three results. Repeat measurements under the same conditions (temperature, SOC, rest time) to compare batteries.

5.3 Backing up your data

- **Simple:** download `ebc_a20.db` in **SD card** when no test is running. The file can be opened with any SQLite tool (e.g. "DB Browser for SQLite").
- **Automatic:** set up the MariaDB backup. Delete runs on the SD card only after verifying that they have been transferred.

6 Troubleshooting

Problem	Possible cause	Remedy
Access point `BatteryTester-…` not visible	No antenna, device not powered or still starting	Check the antenna and power; wait 10–20 s after power-on.
AP password unknown	Randomly generated at first start	Read it from the start-up log over USB-C (serial terminal, press RESET).
Login password not accepted	Password was changed; or the default is still active	Try `ebctester` (default until a password is saved). A forgotten saved password can only be reset by erasing the flash and reinstalling the firmware – this deletes all device settings, the data on the SD card is kept.
Page does not load at 192.168.4.1	Phone switched back to mobile data or another network	Disable mobile data while connected to the AP; check the Wi-Fi connection.
Login keeps returning to the login page	Session expired or device restarted; browser blocks cookies	Log in again; allow cookies for the device address.
Login refused although the password is correct	Maximum of 8 simultaneous sessions reached	Log out in other browsers or restart the device.
RX indicator stays grey	Tester not connected or wiring wrong	Click **Connect**; check TX/RX crossing and GND; check that the tester is switched on.
"Tester mismatch" error	Selected model differs from the connected one	Select the right model in Settings → EBC tester, then Connect.
Start buttons disabled	See the yellow hint below the button	Typical: not connected, waiting for first message, plan running, command pending.
"Database not ready"	No card, card not FAT32, card full or defective	Insert a FAT32 card; check the SD card tab. exFAT cards must be reformatted to FAT32 on a PC.
Times shown as uptime instead of date	Clock not synchronised (no Internet)	Connect the device to Wi-Fi with Internet access.
Plan shows "Interrupted"	Power failure or restart during the plan	Check the battery, click **Reset status**, restart the plan if needed.
DCIR result "invalid"	Sample too late, voltage drop too small, no current	Increase the pulse current, check four-wire sensing and the timing tolerance.
Update check fails	No Internet, clock not yet synchronised, server unreachable	Wait a few minutes after connecting; retry with **Check for updates**.
Settings are not saved	Login password wrong or dialog cancelled	Enter the current login password (not the access-point password) in the confirmation dialog.
Language file not offered	Wrong folder, invalid JSON or wrong code	Place it in `/languages`, check the warning shown in the header, click Reload languages.

If the problem persists, enable the debug log, reproduce the problem and download the log. When reporting a problem on GitHub, include the board, firmware version, steps to reproduce and the log – but never passwords or private data.

Part II – Technical Reference

This part is intended for readers who want to build the hardware themselves, compile the firmware, integrate EBCTester into other systems or understand how it works internally. It covers the circuit, the connectors and parts, firmware configuration, the tester protocol, data storage, interfaces and security.

7 System architecture

7.1 Firmware structure

The firmware is written in C/C++ with Espressif's ESP-IDF 6.0.1 for the ESP32-S3. It is divided into functional components; the main program only initialises services. Networking, updates and SD-card handling are separate, reusable repositories included as Git submodules.

Repository / component	Responsibility
EBCTester	Application: tester control, plans, recording, charts, DCIR, MariaDB backup
`ebc_link`	UARTs, frame parser, model detection and communication with the tester
`test_control / test_store`	Manual control, test plans, batteries, recording of test runs
`resistance_test`	DCIR preparation, load pulse and evaluation
`sqlite3 / battery_database`	SQLite adapter, SD file system layer and database schema
`battery_dashboard`	Measurements tab, charts (Chart.js) and test-run management
`mariadb_sync`	Incremental backup to MariaDB
`ha_mqtt`	Optional Home Assistant MQTT integration
`ui_localization / debug_log`	Additional SD languages; in-memory log and Debug tab
ota-suite	Wi-Fi provisioning, access point, web server and shared UI, login, manual OTA updates, status LED
sd-card-manager	SDMMC/SPI driver, card detection, shared storage lock, file management tab

7.2 Data flow

The tester sends a status message roughly once per second. `ebc_link` checks each message (checksum, model identifier) and passes it to the controller. While a test is active, `test_store` averages a configurable number of messages (default 20, i.e. about one stored measurement every 20 seconds) and writes the measurement in a database transaction to the SD card. Plans and DCIR tests are state machines running in their own tasks; they react to the received messages and send commands to the tester. The web interface only reads data and sends commands through authenticated HTTP requests – it is never needed for a test to continue.

8 Hardware

The EBCTester board (revision 1) is a compact controller board. It contains no measurement circuit of its own – all measurement and power electronics are in the ZKE tester. The board provides the microcontroller, power supply, SD card, real-time clock and a level-shifted serial interface.

> **Note**
> Status of the hardware documentation: schematic revision 1 as of 2 November 2026 (with status LEDs), bill of materials dated 2 November 2026. The PCB layout has not been made yet. Check the GitHub project for newer revisions.

8.1 Block diagram

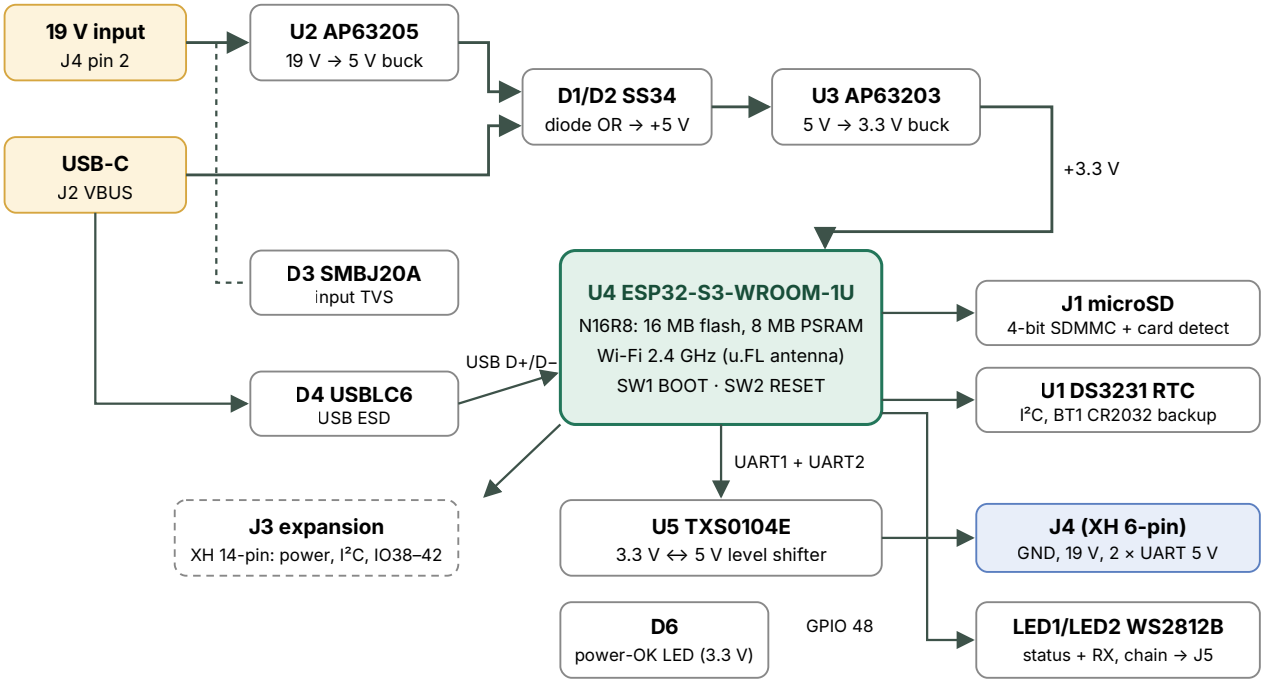


Figure 24: Block diagram of the EBCTester board

8.2 Circuit description

Power supply

The 19 V input (from the EBC-A20 power supply via J4) is protected by the unidirectional TVS diode D3 (SMBJ20A, cathode to +19 V; stand-off voltage 20 V, clamping voltage 32.4 V – below the 35 V absolute maximum input of U2). D3 also protects against reverse polarity by conducting, so the supply should be current-limited or fused. The synchronous buck converter U2 (AP63205, fixed 5 V, up to 2 A) with L1 (4.7 µH) produces 5 V. This output and the USB supply (VBUS) are combined by the Schottky diodes D1 and D2 (SS34) into the **+5 V rail**, which is about 4.6–4.7 V after the diode drop. The board can therefore run from the 19 V input, from USB, or from both at the same time. With USB only, the total current is limited to the 500 mA of the USB port.

A second buck converter U3 (AP63203, fixed 3.3 V) with L2 supplies the ESP32-S3 and all logic from the +5 V rail, so 3.3 V is available from either source. Input and output capacitors follow the Diodes Inc. data sheet: 10 µF/50 V at the 19 V side, 2 × 22 µF at each output, 100 nF bootstrap capacitors. The data sheet suggests 3.9 µH for the 3.3 V version; 4.7 µH is within the permitted range (2.2–10 µH).

Microcontroller and USB

U4 is an ESP32-S3-WROOM-1U-N16R8 module with 16 MB flash and 8 MB octal PSRAM. The "U" variant has no PCB antenna but a u.FL connector – an external 2.4 GHz antenna is mandatory. GPIO 35–37 are used internally by the octal PSRAM and must not be used. The USB-C connector J2 is wired to the native USB of the ESP32-S3 (GPIO 19/20) for flashing and the console; D4 (USBLC6-2SC6) protects the data lines against ESD. R10/R11 (5.1 kΩ on CC1/CC2) identify the board as a USB device to USB-C power supplies.

SW1 (BOOT, GPIO 0, pull-up R12) and SW2 (RESET, EN, pull-up R13 with C16 for a defined power-on reset) allow manual download mode: **hold BOOT, press and release RESET, release BOOT.**

microSD card

J1 (Hirose DM3AT) is connected in 4-bit SD mode. All data and command lines have 10 kΩ pull-ups (R1–R5) as required by the SD specification. The card-detect switch is normally open and closes when a card is inserted; with pull-up R6, the signal `SD_CD` reads **low with a card present**.

Real-time clock

U1 (DS3231SN) is a temperature-compensated RTC (±2 ppm) on I²C with 4.7 kΩ pull-ups (R7/R8). Its INT/SQW output (open drain, pull-up R9) goes to GPIO 14 and can be used as an interrupt or deep-sleep wake-up source. A non-rechargeable CR2032 cell (BT1) on VBAT keeps the time when the board is unpowered. *Firmware 1.4.72 does not use the RTC yet*; the time is obtained by SNTP when Wi-Fi with Internet access is available.

Serial interface

The tester uses 5 V TTL levels, but ESP32-S3 inputs are not 5 V tolerant. U5 (TXS0104E) is a 4-bit bidirectional, auto-direction level shifter between 3.3 V (VCCA) and the +5 V rail (VCCB). It carries UART1 (tester) and UART2 (reserved for the PC dongle) to connector J4. The TXS0104E is suited to push-pull signals at the low baud rate used (9600 Bd).

Status LEDs

LED1 and LED2 are WS2812B RGB LEDs with integrated driver, connected in a chain to GPIO 48 through the series resistor R15 (470 Ω). LED1 shows the operating state, LED2 the reception of tester messages (see section 2.5). Each LED has a 100 nF decoupling capacitor (C17, C18). The WS2812B runs on the +5 V rail, but the ESP32 drives the data line with 3.3 V. To keep this level above the input-high threshold of LED1 (0.7 × V_{DD}), LED1 is supplied through the Schottky diode D5, which lowers its supply by about 0.3 V. LED2 receives the regenerated 5 V signal from LED1. The data output of LED2 is led through R16 (470 Ω) to connector J5, so that further WS2812B LEDs can be added to the chain.

Power-OK LED

D6 (yellow-green, 0603) is connected to +3.3 V through R14 (470 Ω) and lights up as soon as the 3.3 V supply is present – independently of the firmware.

8.3 Connectors, jumper and test points

J4 – tester connector (JST XH, 6-pin, 2.5 mm)

Pin	Signal	Description
1	GND	Ground
2	VIN_19V	19 V supply input (from the EBC-A20 power adapter)
3	UART1_TX_5V	Data from EBCTester to the tester → tester RX
4	UART1_RX_5V	Data from the tester → tester TX
5	UART2_TX_5V	Reserved for the PC dongle bridge
6	UART2_RX_5V	Reserved for the PC dongle bridge

TX/RX are named from the ESP32's point of view. UART2 is initialised by the firmware but the bridge between the manufacturer's PC software and the tester is not implemented yet.

J3 – expansion connector (JST XH, 14-pin, 2.5 mm)

Pin	Signal	Pin	Signal
1	VIN_19V	8	GPIO 42
2	+3.3 V	9	GPIO 41
3	+5 V	10	GPIO 40
4	VBUS	11	GPIO 39
5	RTC_INT (GPIO 14)	12	GPIO 38
6	I2C_SDA (GPIO 15)	13	GND
7	I2C_SCL (GPIO 16)	14	GND

J3 carries the power rails, the I²C bus with the RTC interrupt and five free GPIOs for extensions (e.g. display, temperature sensors). It is not used by the current firmware. Its 3.3 V GPIOs are not 5 V tolerant.

J5 – LED chain extension (JST XH, 3-pin, 2.5 mm)

Pin	Signal	Description
1	+5 V	Supply for external WS2812B LEDs (from the +5 V rail, approx. 4.6–4.7 V)
2	DATA	Data output of LED2 through R16 (470 Ω)
3	GND	Ground

External LEDs draw their current from the +5 V rail. With USB-only supply, keep the total current within the 500 mA of the USB port (a WS2812B draws up to approx. 60 mA at full white).

JP1 – solder jumper "Custom_Settings" (GPIO 47)

A three-pad solder jumper that ties GPIO 47 to +3.3 V or GND. It is reserved for a hardware configuration option (for example a board variant or a default setting read at start-up) and is not evaluated by firmware 1.4.72. As delivered, pads 1 and 2 are bridged (footprint "Bridged12"), so GPIO 47 reads high; cut the bridge and close pads 2–3 for low. GPIO 47 is not a strapping pin, so either position is safe for booting. When JP1 is open, the firmware should enable an internal pull resistor to avoid a floating input.

Test points

TP1–TP10 are round SMD pads (1.5 mm diameter) for probes; they are not separate components.

Test point	Signal	Expected value
TP3	19 V input	approx. 19 V (when supplied via J4)
TP4	5 V buck output (before diode)	5.0 V
TP1	+5 V rail	4.6 – 4.7 V
TP6	USB VBUS	5 V with USB connected
TP2	+3.3 V	3.3 V
TP5	GND	reference
TP7 / TP8	BOOT (GPIO 0) / RESET (EN)	3.3 V idle, 0 V when pressed
TP9 / TP10	RXD0 (GPIO 44) / TXD0 (GPIO 43)	UART0: boot messages of the ESP32 (3.3 V level, 115200 Bd)

GPIO assignment of board revision 1

Function	GPIO	Function	GPIO
UART1 TX / RX (tester)	10 / 11	SD CLK / CMD	6 / 7
UART2 TX / RX (dongle)	8 / 9	SD D0 / D1 / D2 / D3	5 / 4 / 18 / 17
I²C SDA / SCL (RTC)	15 / 16	SD card detect (low = card)	13
RTC INT/SQW	14	LED1/LED2 data (WS2812B)	48
JP1 Custom_Settings	47	Strapping pins, keep free	3, 45, 46
USB D− / D+	19 / 20	Expansion J3	38 – 42
BOOT	0	PSRAM (do not use)	35 – 37
UART0 RXD0 / TXD0 (TP9 / TP10)	44 / 43	Not connected (no-connect)	1, 2, 12, 21

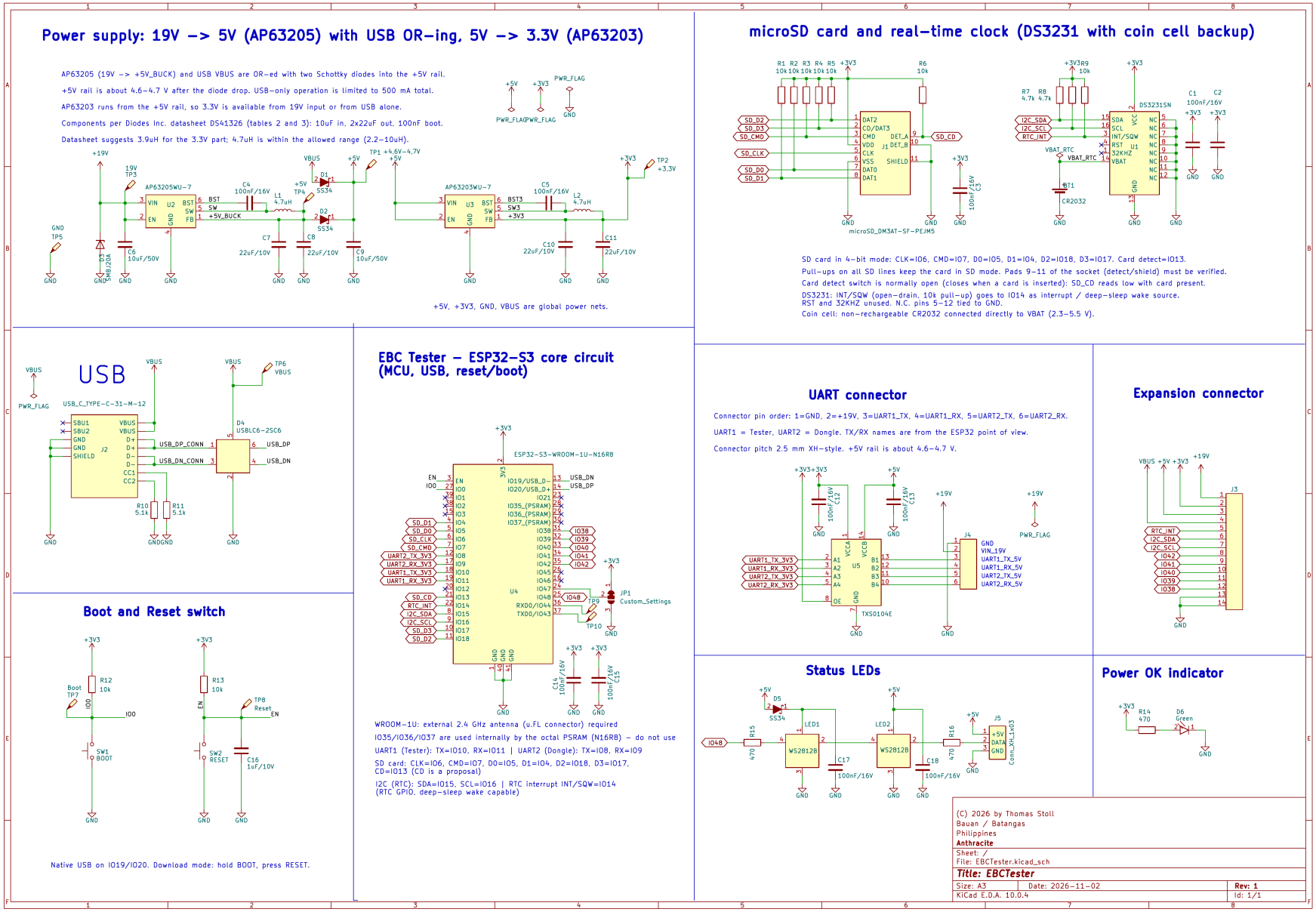


Figure 25: Schematic of the EBCTester board, revision 1 (KiCad 10). The PDF is published in the repository as docs/EBCTester_Schematic.pdf.

8.4 Bill of materials

26 line items, 57 components (53 SMD, 4 THT). All parts were checked at LCSC on 1 and 2 November 2026; check stock before ordering. Ready-to-upload BOM files for JLCPCB and PCBWay are in the hardware repository.

Ref.	Qty	Value / function	Package	Manufacturer / MPN	LCSC
U1	1	RTC with TCXO, I²C	SOIC-16W	Analog Devices DS3231SN#T&R	C9866
U2	1	Buck 19 V → 5 V, 2 A	TSOT-23-6	Diodes AP63205WU-7	C2071056
U3	1	Buck 5 V → 3.3 V, 2 A	TSOT-23-6	Diodes AP63203WU-7	C780769
U4	1	ESP32-S3 module, 16 MB/8 MB, u.FL	Module	Espressif ESP32-S3-WROOM-1U-N16R8	C3013946
U5	1	Level shifter 3.3 V / 5 V	TSSOP-14	TI TXS0104EPWR	C44955
LED1, LED2	2	RGB LED with driver, status / RX	5050	Worldsemi WS2812B-B/W	C114586
J1	1	microSD socket, card detect	SMD	Hirose DM3AT-SF-PEJM5	C114218
J2	1	USB-C receptacle (USB 2.0)	SMD	HRO TYPE-C-31-M-12	C165948
J3	1	Header 14-pin, 2.5 mm	THT	BOOMELE XH-14A	C20200
J4	1	Header 6-pin, 2.5 mm	THT	BOOMELE XH-6A	C5663
J5	1	Header 3-pin, 2.5 mm (LED chain)	THT	BOOMELE XH-3A	C2316
D1, D2, D5	3	Schottky 40 V / 3 A (OR-ing; LED1 supply)	SMA	MDD SS34	C8678
D3	1	TVS 20 V, 600 W	SMB	TWGMC SMBJ20A	C726828
D4	1	USB ESD protection	SOT-23-6	ST USBLC6-2SC6	C7519
D6	1	LED yellow-green, power OK	0603	XINGLIGHT XL-1608SYGC-04	C965806
L1, L2	2	4.7 µH power inductor	6 × 6 mm	Bourns SRN6045TA-4R7M	C2044594
SW1, SW2	2	Tactile switch BOOT / RESET	5.1 × 5.1 mm	XKB TS-1187A-B-A-B	C318884
BT1	1	CR2032 holder	THT	MYOUNG BS-07-A1BJ001	C2979167
C1–C5, C12–C15, C17, C18	11	100 nF 16 V X7R	0402	TDK C1005X7R1C104KT000F	C338032
C6, C9	2	10 µF 50 V X7S	1206	Samsung CL31Y106KBKVPNE	C2803389
C7, C8, C10, C11	4	22 µF 10 V X5R	0805	Samsung CL21A226KPCLRNC	C318688
C16	1	1 µF 10 V X7R	0402	Venkel C0402X7R100-105KNP	C3846824
R1–R6, R9, R12, R13	9	10 kΩ 1 %	0402	Vishay CRCW040210K0FKED	C71617
R7, R8	2	4.7 kΩ 1 %	0402	Vishay RCC04024K70FKED	C1858949
R10, R11	2	5.1 kΩ 1 %	0402	Vishay CRCW04025K10FKTD	C4333224
R14, R15, R16	3	470 Ω 5 % (power LED, LED data, chain output)	0402	Vishay CRCW0402470RJNED	C482183

Additional parts (not assembled)

#	Part	Description	LCSC
A1	Antenna cable kinghelm KH-IPEX-SMA-W-150	u.FL (IPEX1) to SMA female bulkhead, 150 mm, with nut	C2683343
A2	Antenna DreamLNK W10-2.4G	2.4 GHz, 3 dBi, SMA male, foldable, approx. 105 mm	C381198
A3	CR2032 coin cell	Any brand; buy locally (lithium cells are restricted for shipping)	–
A4	Mating cable for J4	XH 2.5 mm 6-pin housing BOOMELE XH-6Y (C10389) with crimp contacts XH-T (C5670), 6 × UL1007 24 AWG, 150 mm, other end stripped and tinned	custom cable
A5	Mating cable for J3	XH 2.5 mm 14-pin housing BOOMELE XH-14Y (C38729) with crimp contacts XH-T (C5670), 14 × UL1007 24 AWG, 150 mm, other end stripped and tinned	custom cable

- **Antenna:** mount the SMA bulkhead of A1 in the enclosure wall (hole approx. 6.5 mm), plug the u.FL end onto U4 and screw on A2. Use standard SMA, not RP-SMA. Alternatives: cable lengths 120 mm (C2683342) or 200 mm (C2683345); antenna BAT WIRELESS BW2.4JWX195-13KJ (195 mm, 8 dBi, C497018).
- **Ordering:** A1 and A2 can be ordered at LCSC and shipped together with a JLCPCB assembly order.
- **Mating cables A4/A5:** ready-made XH cables with 150 mm wires are not a standard stock item. Order them through the LCSC Custom Cables service (minimum quantity 1, quotation within 24 hours): housing and contacts as listed, wire UL1007 24 AWG, length 150 mm, all positions populated, other end stripped and tinned. Pin 1 of the housing must match pin 1 of the connector. Custom cables can be combined with the LCSC/JLCPCB shipment. Off-the-shelf alternative for J4 only (200 mm): BXCONN XH6P-24AWG-200-A1T (C53413416).
- **CR2032:** insert with "+" facing up.
- A microSD card is not included.

9 Firmware: installation and configuration

9.1 Installing a release (no build environment)

Each GitHub release contains a file `EBCTester-<version>-full-install.bin` with bootloader, partition table and application. Flash it at address `0x0`, for example with Espressif's esptool:

```
esptool.py --chip esp32s3 --port COM5 write_flash 0x0 EBCTester-1.4.72-full-install.bin
```

If the board is not recognised, start it in download mode (hold BOOT, press RESET). Later updates can be installed over Wi-Fi (section 4.4). The module must have **16 MB flash**.

> **Caution**
> Release binaries use the firmware's default pin assignment (section 9.3). Check that it matches your hardware before using a release image on the EBCTester board.

9.2 Building from source

Requirements: VS Code with the ESP-IDF extension, **ESP-IDF 6.0.1**, Git and an Internet connection for the first build (managed components and the SQLite source are downloaded and hash-checked).

```
git clone --recurse-submodules https://github.com/Gemelon/EBCTester.git
# existing checkout:
git submodule update --init --recursive
```

1. Open the project in VS Code and select the ESP-IDF environment and target **esp32s3**.
2. Review board, flash, PSRAM, SD mode and UART pins in **menuconfig** (see below).
3. Build with **ESP-IDF: Build your project**.
4. For the first installation, flash bootloader, partition table and application over USB.

A ZIP download of the repository does not contain the submodules. Do not run `git submodule update --remote` without reviewing the changes – the main repository records the tested versions. `sdkconfig.defaults` provides portable defaults; an existing local `sdkconfig` takes priority. Size optimisation (`-Os`) is the default because the Xtensa GCC 15.2 toolchain crashes with `-Og` in an ESP-IDF driver.

Important menuconfig options

Menu	Option	Default
EBCTester → EBC battery tester UARTs	Tester UART TX / RX GPIO, baud rate	17 / 18, 9600
	Dongle UART TX / RX GPIO, baud rate	8 / 9, 9600
EBCTester → test control	Averaging length (frames per stored measurement)	20
EBCTester → test store	Device ID, max. pending samples	EBC-A20, 3000
EBCTester → Home Assistant MQTT	Enable MQTT component	off
SD Card Manager	Bus mode (4-bit SDMMC / SPI), GPIOs, clock	SDMMC, 10 MHz
	SDMMC CLK / CMD / D0 / D1 / D2 / D3	14 / 15 / 2 / 4 / 5 / 6

Menu	Option	Default
	Card-detect GPIO (active high)	7
OTA Suite network	Access-point password (8–63 characters), SNTP server	empty = random, pool.ntp.org
OTA Suite web interface	Initial web login password	ebctester
OTA Suite update checks	Firmware manifest URL	placeholder
OTA Suite firmware updates	Allow unencrypted HTTP OTA	off
	Require login password for settings changes	on
OTA Suite status LED	Enable WS2812 LED, GPIO	on, 48

9.3 Firmware defaults and board revision 1

The firmware defaults were chosen for a development setup with an ESP32-S3-DevKitC-1. The EBCTester board revision 1 uses a different pin assignment. Before using the board, the following settings have to match:

Signal	Board rev. 1	Firmware default	Action
UART1 TX / RX (tester)	GPIO 10 / 11	GPIO 17 / 18	set in menuconfig
UART2 TX / RX (dongle)	GPIO 8 / 9	GPIO 8 / 9	matches
SD CLK / CMD	GPIO 6 / 7	GPIO 14 / 15	set in menuconfig
SD D0 / D1 / D2 / D3	GPIO 5 / 4 / 18 / 17	GPIO 2 / 4 / 5 / 6	set in menuconfig
SD card detect	GPIO 13, **low** = card present	GPIO 7, **high** = card present	GPIO in menuconfig; polarity needs a firmware option
Status LED	WS2812B chain on GPIO 48 (LED1 status, LED2 RX)	one WS2812 on GPIO 48, update animation only	matches electrically; status/RX functions planned
RTC DS3231	I²C GPIO 15/16, INT GPIO 14	not used	future extension

> **Warning**
> The card-detect input of firmware 1.4.72 is active-high only; with the board's active-low switch the card would be reported as missing. This must be resolved (firmware option or hardware change) before the board is used.

9.4 Partition table

Name	Type	Offset	Size	Use
nvs	data/nvs	0x9000	24 KB	Settings (Wi-Fi, passwords, model, plan, battery …)
otadata	data/ota	0xF000	8 KB	Active firmware slot
phy_init	data/phy	0x11000	4 KB	RF calibration
ota_0 / ota_1	app	0x20000 / 0x590000	2 × 5.4 MB	Two firmware slots for safe updates
storage	data	0xB00000	5 MB	Reserved, currently unused

A changed partition table cannot be installed over the air; it requires flashing over USB. Back up settings and data before repartitioning.

9.5 Status LED in firmware 1.4.72

Firmware 1.4.72 drives one WS2812 RGB LED (on the EBCTester board: LED1; on the ESP32-S3-DevKitC-1: GPIO 48 or 38) and shows the update state: slow colour cycle = normal operation, blue pulse = firmware download, yellow pulse = verification, solid green = waiting for restart, red blinking = update failed. Brightness is limited to 24/255. LED2 is not driven yet. The status and RX functions described in section 2.5 will replace the colour cycle in a coming firmware version.

10 Communication with the tester

10.1 Serial interface

UART1 is connected to the tester with **9600 baud, 8 data bits, odd parity, 1 stop bit (8O1)** at 5 V TTL level. Frames start with `0xFA` and end with `0xF8`. Status frames are 19 bytes long. An XOR checksum over bytes 1–16 is verified before any data is used. The EBC-A20 transmits checksums in the range 0xF0–0xFF with the upper nibble cleared, to avoid the frame markers; this variant is also accepted.

Set-points are transmitted as two bytes in base 240 (*value = high × 240 + low*) with a resolution of 0.01 A or 0.01 V. The range is sufficient for 50 A; the 10 mA resolution has not been verified on real A50L hardware yet. The discharge command also carries a time limit in whole minutes.

10.2 Model detection

Byte 16 of each status frame contains the model identifier (normalised as raw value modulo 100). For status bytes of at least 100, bytes 10 and 11 contain the tester's firmware version (*B10 × 240 + B11*), which distinguishes variants with the same identifier.

Ident (dec / hex)	Firmware version	Model	Enabled
5 / 0x05	302 / other	EBC-A05+ / EBC-A05	no
6 / 0x06	310 / other	EBC-A10+ / EBC-A10H	no
9 / 0x09	311 / other	EBC-A20+ / **EBC-A20**	A20 only
10 / 0x0A	311 / other	**EBC-A50L** / **EBC-A40L**	yes
20 / 0x14	310 / other	EBD-M10 / EBD-A10H	no
25 / 0x19	310 / 315 / other	EBD-M03+ / EBD-M05 / EBD-M03	no
38 / 0x26	310 / 311 / other	EBD-A100L / EBC-A100L / EBD-A100	no

While the firmware version of an ident-10 tester is not yet known, it is shown as "EBC-A40L/A50L (Version unknown)" and the conservative A40L limits are used. The model check runs on every valid frame. On a mismatch with the selected model, the connection and measurement state are cleared, pending commands are discarded, Disconnect is sent and automatic reconnection is suppressed.

10.3 Operating modes

Code	Mode	Code	Mode	Code	Mode
0x00	D-CC stopped	0x0A	D-CC active	0x14	D-CC completed
0x01	D-CP stopped	0x0B	D-CP active	0x15	D-CP completed
0x02	C-CV stopped	0x0C	C-CV active	0x16	C-CV completed

11 Data storage

11.1 SQLite database on the SD card

The database file is `/sdcard/ebc_a20.db`. It is created when a card is mounted; if the device starts without a card, it is created as soon as one is inserted. The card is never formatted and an existing database is never deleted. A file without a valid SQLite header is preserved under the name `ebc_a20.db.invalid-XXXXXXXX` before a new database is created.

Table	Content
`ebc_battery`	Batteries (label, manufacturer, serial number, date, nominal capacity, notes)
`ebc_test_plan` / `ebc_test_plan_step`	Saved test plans and their steps
`ebc_test_run`	Test runs (battery, plan, start/end, status, current step, cycle, firmware, notes, display name)
`ebc_run_segment`	Segments of a run (mode, step, cycle, set-points)
`ebc_measurement`	Measurements (time UTC and uptime, current, voltage, capacity …)
`ebc_resistance_result`	Three DCIR results per test (times, U1/I1, U2/I2, mΩ, validity)
`ebc_schema_metadata`, `ebc_sync_state`	Schema version; MariaDB synchronisation progress

The views `ebc_measurement_view` and `ebc_resistance_result_view` make evaluation easier. SQLite runs with `synchronous=FULL`, real `fsync()`, foreign keys and an integrity check at start-up.

11.2 Schema versions

Schema	Firmware	Change
1	–	Base tables (corresponds to MariaDB migrations 001–005)
2	DCIR support	Table `ebc_resistance_result`
3	1.4.67 and later	Column `display_name` for custom run names

Migrations run automatically in a transaction at the first start with a card. Older firmware refuses a newer schema: keep a database backup before an update if you might need to downgrade.

Time stamps are stored in UTC and require a synchronised clock (SNTP). Without it, the device uptime is recorded instead; no calendar time is invented.

11.3 MariaDB backup

The MariaDB synchroniser copies plans, steps, batteries, runs, segments, measurements and DCIR results to an existing MariaDB schema (e.g. `ebc_a20`). Progress is tracked per run; only new data is sent in batches of 200 rows with `INSERT IGNORE`.

- The base schema (migrations 001–005) must exist on the server. The repository contains migration `sql/006_resistance_results_mariadb.sql` for DCIR results, not a complete initial schema.
- The user needs a `mysql_native_password` account with SELECT/INSERT/UPDATE on `ebc_test_plan`, `ebc_battery`, `ebc_test_run`, additionally DELETE on `ebc_test_plan_step`, and INSERT on `ebc_run_segment` and `ebc_measurement`, plus rights on the DCIR result table.

- The connection is plain TCP without TLS.

Example query of the DCIR results:

```
SELECT battery_label, test_started_at_utc, requested_seconds,
       actual_seconds, resistance_mohm, valid, error
FROM ebc_resistance_result_view
ORDER BY test_started_at_utc DESC, run_id, result_number;
```

12 Interfaces

12.1 HTTP API

The web interface uses a JSON API that can also be used by your own scripts. With browser login enabled, all endpoints require a login session (cookie); POST requests additionally require the CSRF token from `GET /api/status` in the header `X-CSRF-Token`. Saving settings also requires `X-Admin-Password` when the password confirmation is enabled.

Endpoint	Method	Purpose
`/api/auth/login, /api/auth/logout`	POST	Login (JSON user name/password, header `X-Web-Login: 1`) and logout
`/api/status`	GET	Device, network and update status, CSRF token, `sessionDays`, `loginEnabled`
`/api/auth/settings`	POST	Login settings: `sessionDays` (1/7), `loginEnabled`, optional new `password`
`/api/tester/status`	GET	Live values, connection, model and limits, plan, recorder
`/api/tester/settings, /api/tester/catalog`	GET	Manual values, battery, plan; saved batteries and plans
`/api/tester/command`	POST	connect, disconnect, stop, auto_connect, charge, discharge_cc, discharge_cp
`/api/tester/plan, /api/tester/plan/action`	POST	Save plan; start, abort, reset, load_db, save_db
`/api/tester/battery, /api/tester/model`	POST	Battery actions; tester model selection
`/api/tester/resistance(/history)`	GET/POST	DCIR configuration, status, start/abort; history
`/api/db/runs, /api/db/run?id=`	GET	Batteries and latest 300 runs; run details and segments
`/api/db/measurements?id=&max=`	GET	Series `[segment, uptimeMs, epochMs, I, U, Ah]`, reduced to max points (200–6000)
`/api/test-runs(/rename|/delete)`	GET/POST	Test-run management
`/api/mariadb/…, /api/mqtt/…, /api/debug/…`	GET/POST	Status and settings of the optional services
`/api/sd/…, /api/updates/check, /api/ota`	GET/POST	SD card file operations; update check and installation

12.2 Home Assistant MQTT

The MQTT component must be enabled at build time (menuconfig) and is configured in the web interface. On connection the device publishes a retained discovery document to `homeassistant/device/ebctester_<MAC>/config`, availability (`online` / `offline` as Last Will) to `<prefix>/availability`, and every recorded measurement (voltage, current, capacity) as a non-retained QoS 0 message to `<prefix>/state`. Mode, plan status and battery entities are already announced but do not yet receive values.

Commands are accepted on `<prefix>/command` as a JSON object:

```
{"command":"start_plan"}        (only if remote start is allowed)
{"command":"stop"}
{"command":"abort"}
{"command":"select_plan","name":"Plan name"}
{"command":"select_battery","name":"Battery label"}
```

MQTT cannot set free set-points or create database entries. Invalid commands are ignored. Failed connections are retried with increasing intervals from 1 to 60 seconds.

13 Firmware updates and publishing

13.1 Update procedure

- The device downloads only the manifest `firmware.json` (after Wi-Fi connects, every six hours, when the UI is opened). HTTPS checks wait for a synchronised clock.
- Versions must be strictly numeric `major.minor.patch` and newer than the installed one.
- Manifest and image must be delivered directly (HTTP 200, no redirects). HTTPS validates certificate chain, host name and dates. HTTP is only possible with an explicit build option for a trusted test network.
- The image is streamed into the inactive slot; exact size and SHA-256 are checked before it is selected for booting. Project name and target must match.
- After restart the new firmware must run for 10 seconds; an earlier reset or crash rolls back to the previous version.

13.2 Manifest format

```
{
  "project": "EBCTester",
  "target": "esp32s3",
  "version": "1.4.73",
  "url": "https://updates.example.org/BatteryTester/EBCTester-1.4.73.bin",
  "size": 2734352,
  "sha256": "<64 lower-case hex digits>",
  "notes": "Release notes"
}
```

All seven fields are mandatory; the manifest must be smaller than 2048 bytes, notes at most 512 bytes. The image URL must be in the same directory as the manifest and named `<project>-<version>.bin`.

13.3 Publishing your own builds

Publishing is disabled in a fresh checkout. It is enabled with a local, Git-ignored file `CMakeUserConfig.cmake` (see README). A successful build then creates the immutable versioned image, a compatibility copy and finally the manifest, optionally uploaded by SFTP. A published version is never overwritten with different content – increase the version first. Always distribute the license directory together with the firmware.

14 Security and limitations

14.1 Security

- **Login:** user `admin` with the web login password (initially `ebctester`, set by `CONFIG_OTA_SUITE_WEB_PASSWORD`; a password saved in Login Settings is stored in NVS namespace `web_auth` and takes precedence). Browser login can be disabled; then status, data and controls are accessible without a session. Sessions use a random 256-bit cookie (`HttpOnly`, `SameSite=Lax`), last exactly 1 or 7 days and are held in RAM (a restart ends them). At most 8 sessions at a time; failed logins are delayed by one second.
- **CSRF:** all changing requests require a CSRF token; settings additionally require the current login password in the header `X-Admin-Password`, independent of whether browser login is enabled (configurable at build time). Password or login-policy changes invalidate all sessions.
- **Transport:** the web interface, MQTT and MariaDB are unencrypted. Use only trusted networks, never forward port 80 to the Internet.
- **Storage:** NVS and flash are not encrypted by default; protect physical access and the USB port (the start-up log shows the access-point password). Change the shared initial login password `ebctester` on first use.
- **Firmware:** SHA-256 protects against corruption, not against deliberate replacement. Secure Boot, signed images and flash encryption are separate deployment decisions.

14.2 Known limitations (firmware 1.4.72)

- The UART2 bridge for the manufacturer's PC software is not implemented.
- The DS3231 real-time clock is not used yet; date and time come from SNTP only.
- Only EBC-A20, EBC-A40L and EBC-A50L can be controlled; physical tests with the A50L are pending.
- No battery-chemistry profiles and no temperature measurement.
- MQTT publishes measurements only; status entities do not yet receive values.
- Board revision 1 requires firmware adjustments (section 9.3).
- The status and RX functions of LED1/LED2 are not implemented yet.
- A forgotten login password can only be reset by erasing the flash.

14.3 License

Copyright 2026 Thomas Stoll. EBCTester, OTA Suite and SD Card Manager are licensed under the Apache License 2.0 – commercial use, modification and sale are permitted under its terms; its warranty and liability disclaimers apply. Third-party components keep their own licenses: Bootstrap 5.3.3 and Chart.js 4.4.9 (MIT), SQLite (public domain), ESP-IDF (Apache 2.0) and others, listed in `THIRD_PARTY_NOTICES.md`. EBCTester is an independent project and not an official ZKETECH product; product names are trademarks of their respective owners.

Appendix A – Language files

Additional user-interface languages are loaded from the folder `/languages` on the SD card. A complete template is `sd_card/languages/en-custom.json` in the repository; a partial Filipino (Tagalog) example is `fil.json`.

1. Copy the template, e.g. as `fr.json`.
2. Set `"code": "fr"` and `"name": "Français"`.
3. Translate only the values in `"strings"`; keep all keys unchanged.
4. Keep placeholders such as `{p0}`, `%u`, `%s` (printf placeholders in the same order).
5. Save as UTF-8 without BOM, upload to `/languages/fr.json` (e.g. in the SD card tab), click **Reload languages** and select the language in the header.

```
{
  "version": 1,
  "code": "fr",
  "name": "Français",
  "strings": {
    "language.title": "Langue",
    "...": "..."
  }
}
```

Missing keys fall back to English. Files named `en.json` or `de.json` replace the built-in texts while present. Limits: max. 16 files, 128 KB per file, 1024 entries, 4096 bytes per value, 80 bytes for the name. Language codes use lower-case letters, digits and hyphens (max. 24 characters), e.g. `fr`, `en-gb`.

Appendix B – Glossary

AP	Access point – the device's own Wi-Fi network for setup and fallback access.
C-CV	Charging with constant current, then constant voltage (CC/CV).
D-CC / D-CP	Discharging with constant current / constant power.
DCIR	Direct-current internal resistance, measured with a load pulse.
CSRF	Cross-site request forgery; the token protects against requests from foreign web pages.
Frame	One message between EBCTester and the tester.
NVS	Non-volatile storage in the ESP32 flash for settings.
OTA	Over-the-air – firmware update over Wi-Fi.
Segment	Section of a test run with one operating mode.
SNTP	Simple Network Time Protocol – time synchronisation over the Internet.
SOC	State of charge in percent.
Test run	Recording of one manual operation, plan or DCIR test with all its measurements.